

ASTE-331a: Spacecraft Systems Engineering

Avionics

C&DH and FSW

Instructors: Jim Chase, Danielle Marsh

TAs: Daejah Dandrich, Luis Diaz, Liam Kerrigan, Emerson Lo

October 3rd, 2025

Avionics, Part 1 – Overview, HW and SW Architectures, and Examples

Overview

Avionics

Introduction

- **Overview**

- HW Overview
- FSW Overview
- Examples

Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault
mgmt

Design Process

Case Studies

- **General structure:**
 - In this lecture, we will walk through an overview of avionics hardware and software, and we will specifically talk about avionics in the context of:
 - Homework 9 (SC sim verification), in which you will write a FSW procedure to test a set of spacecraft commands and telemetry
 - Homework 10 (Avionics design), in which you will be asked to select avionics hardware for various missions
 - This lecture will be split into three parts:
 - Part 1: Avionics hardware and software architectures and examples
 - Part 2: Avionics functions, spacecraft software simulation
 - Part 3: Avionics design process, case studies

Overview

Avionics

Introduction

• Overview

- HW Overview
- FSW Overview
- Examples

Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

• Function

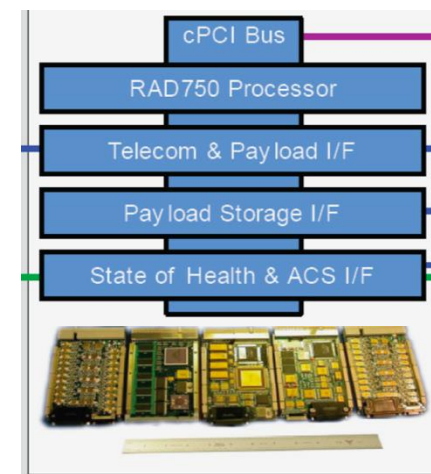
- Provide spacecraft command and control, internal data handling, fault management. This includes FSW with boot/reset functionality. **The brains of the spacecraft!**
- Primary drivers include:
 - Time critical activities – e.g., orbit insertion, landing, etc
 - Environment – thermal, radiation, etc
 - Number, interface type, complexity, and data production of spacecraft devices (e.g., ACS, payload)
 - Fault tolerance requirements, including “up-time”
- Also called Flight Computer

• Common Components

- Flight computer electronics box/chassis
 - Processor (eg, Rad750)
 - Payload interface cards
 - Data interface cards
 - Memory
 - Power converters
- Depending on complexity, the above functions may be in a single box or multiple boxes
- Independent electronics for health monitoring of flight computer (“lizard brain”)

• GRAIL Example

- Rad750 processor
- Interface cards, converters, backplane
- Chassis assembly
- Total = ~5 kg



• Key Trades & Analyses

- Hardware vs software to host functions (timing, configurability, etc)
- Interface types (1553, LVDS, RS-422, etc)
- Onboard memory storage & CPU utilization
- Radiation shielding needs
- Heritage from prior systems
- Redundancy and cross-strapping

• Key Parameters

- Mass, power, & cost

Acronyms and Terminology

Avionics

Introduction

• Overview

- HW Overview
- FSW Overview
- Examples

Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

- **Backplane:** Circuit board that connects boards of a computer and enables them to communicate with each other (cPCI is a common backplane)
- **Chassis:** Structural enclosure to house electronics
- **Command:** An instruction sent to the spacecraft that it then processes to tell it what to do
- **Downlink:** The process of sending data from the spacecraft to the ground
- **Electrically Erasable Programmable Read Only Memory (EEPROM):** Non-volatile memory that can be written to more than once
- **Flight Computer (FC):** Main spacecraft control electronics (i.e., the “brains” of the spacecraft)
- **Flight Software (FSW):** The code that resides within the flight computer to execute most spacecraft control (e.g., ACS, power control, thermal control)
- **Programmable Read Only Memory (PROM):** Non-volatile memory that can only be written to once
- **Random Access Memory (RAM):** Volatile memory that provides temporary data storage for quick read and write access
- **Remote Engineering Unit (REU):** “Lizard brain” avionics hardware that monitors health of main flight computer (JPL-specific terminology)
- **Single Board Computer (SBC):** Hosts the processor and FSW
- **Surom:** Firmware on processor used to boot and configure the flight computer
- **NAND Memory:** Non-volatile flash memory with high density
- **NOR Memory:** Non-volatile flash memory with low density
- **Nonvolatile Memory (NVM):** Computer memory that retains data after a power cycle, often used for long term data storage
- **Parameter:** A settable state that the spacecraft can use as part of its logic (e.g., wait times, error thresholds)
- **Sequence:** A file of commands uploaded from the spacecraft to ground that can be executed in order and with logic (e.g., conditional checks)
- **System Modes:** The overall set of configurations within which the spacecraft can operate, which includes a defined set of hardware states (e.g., nominal mode = IMU on, star tracker on, science instruments on, safe mode = IMU on, star tracker off, science instruments off)
- **Telemetry:** Data collected from spacecraft devices and sent to the ground for interpretation (can be housekeeping, science, etc)
- **Uplink:** The process of sending commands from the ground to the spacecraft
- **Volatile Memory:** Computer memory that loses data after a power cycle, often used for real-time task execution that requires quick access speeds
- *Note: others to be added for parts 2 and 3*

Hardware Overview

Avionics

Introduction

- Overview
- **HW Overview**
- FSW Overview
- Examples

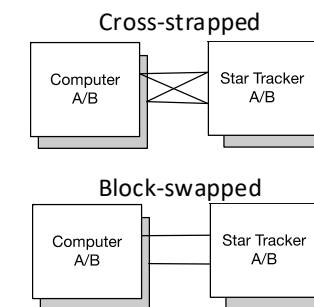
Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

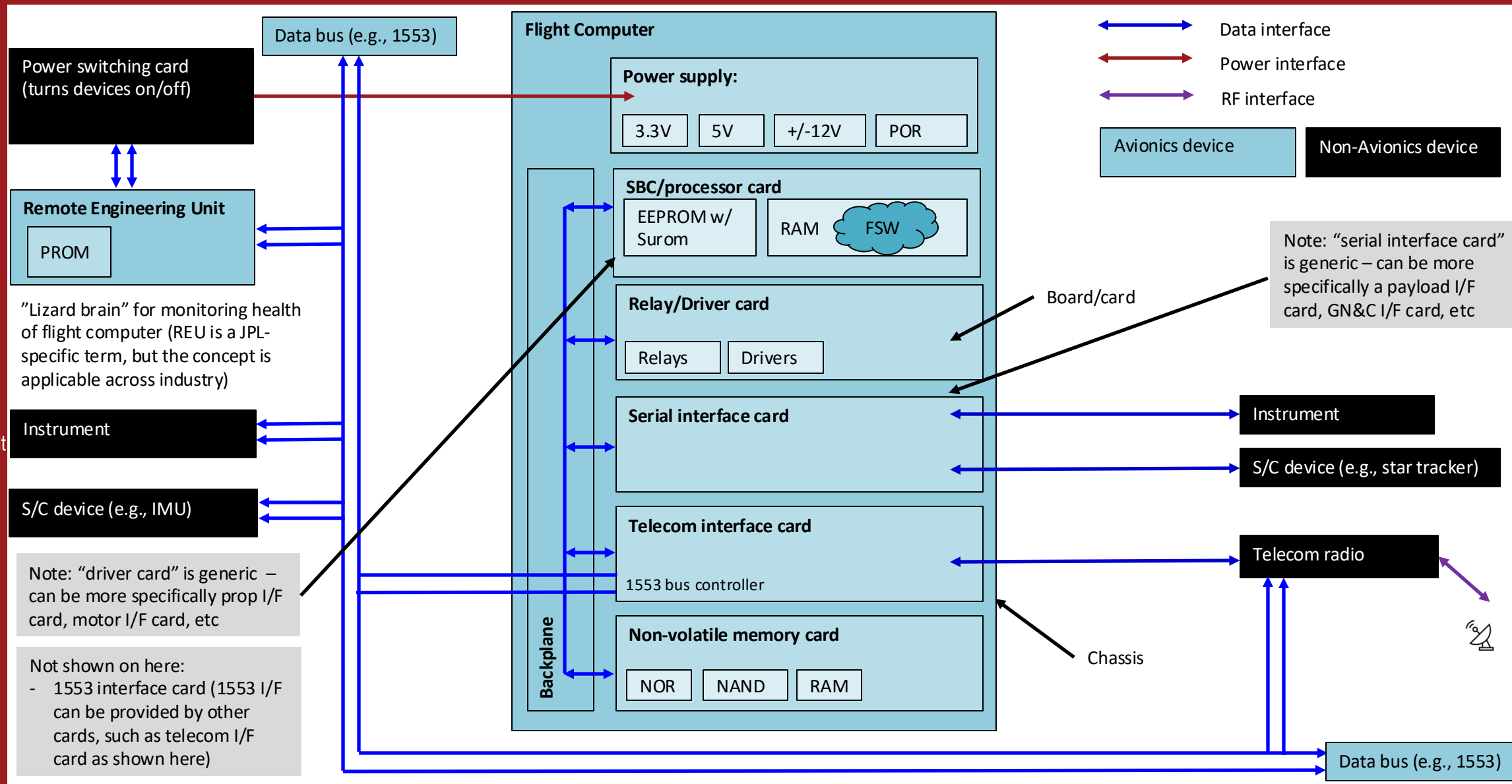
Case Studies

- **Avionics hardware contains the chassis (housing) and cards (circuit boards) necessary to interface with spacecraft devices execute the various avionics functions, including:**
 - **Processing** – executing all functions (including those below) necessary for spacecraft operation
 - **Device intercommunication** – sending commands and receiving data from all devices (e.g., ACS devices, instruments, etc)
 - **Uplink and management of sequences/commands** – receiving and decoding commands and files sent from the ground
 - **Downlink and management of data/telemetry** – storing and formatting data collected by devices and sending to ground
 - **Time management** – maintaining spacecraft clock and distributing time to devices that need it
 - **Autonomous behaviors** – includes management of modes, attitude determination & control, power, thermal, etc
 - **Fault management** – detecting and responding to off-nominal events (e.g., low battery state of charge, loss of comm.)
- **Flight computer hosts central processor, but individual boards will also have their own microcontrollers (e.g., FPGAs) for logic required to interface with other functions/boards**
- **Avionics hardware robustness and fault tolerance considerations include:**
 - Single string vs redundant
 - Block-swapped vs cross-strapped
 - Cold spare vs warm spare operation
 - Independent “Lizard brain” device to monitor health of flight computer



Hardware Architecture Overview

- Avionics**
- Introduction
 - Overview
 - **HW Overview**
 - FSW Overview
 - Examples
- Functions**
- Processing
 - Intercomm
 - Uplink/command
 - Downlink/data
 - Time mgmt.
 - Autonomy & Fault mgmt
- Design Process**
- Case Studies**



Hardware Architecture Overview

See Avionics design HW for more complete descriptions and additional card types/examples

Avionics

Introduction

- Overview
- **HW Overview**
- FSW Overview
- Examples

Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

Avionics hardware	Role	Related avionics functions
Power supply	• Receives primary bus power (typically 31V) • Provides secondary power conversion and distribution for flight computer	• N/A – housekeeping function
SBC/Processor card	• Hosts CPU and flight software • Provides boot and initialization function • Provides spacecraft clock 	• Processing • Device intercommunications • Uplink and command management • Downlink and data management • Time management • Autonomous behaviors • Fault management
Relay/Driver card (could be split up)	• Provides hardware relays that can be operated independent of FSW (e.g., to reset flight computer, to select FSW image) • Provides drivers for propulsion hardware (e.g., latch valves), motor control hardware (e.g., actuators), sensor interfaces (e.g., pressure, temp), etc	• Processing • Device intercommunications
Serial interface card	• Provides serial (point to point) data interface between flight computer and spacecraft devices – e.g., ACS devices, instruments, etc (can be tailored for each type of interface)	• Device intercommunications
Telecom interface card	• Provides interface for uplink/downlink between spacecraft and ground • Provides uplink command decoding and downlink data encoding • Can be used as 1553 bus controller	• Device intercommuncions • Uplink and command management • Downlink and data management
NVM card	• Provide bulk data storage for housekeeping data and payload data • Provides data storage of redundant FSW images	• Downlink and data management • Fault management
Remote engineering unit	• Provides independent health checking and redundancy management (i.e., swaps) of flight computer	• Fault management

Flight Software Architecture Overview

Avionics

Introduction

- Overview
- HW Overview
- **FSW Overview**

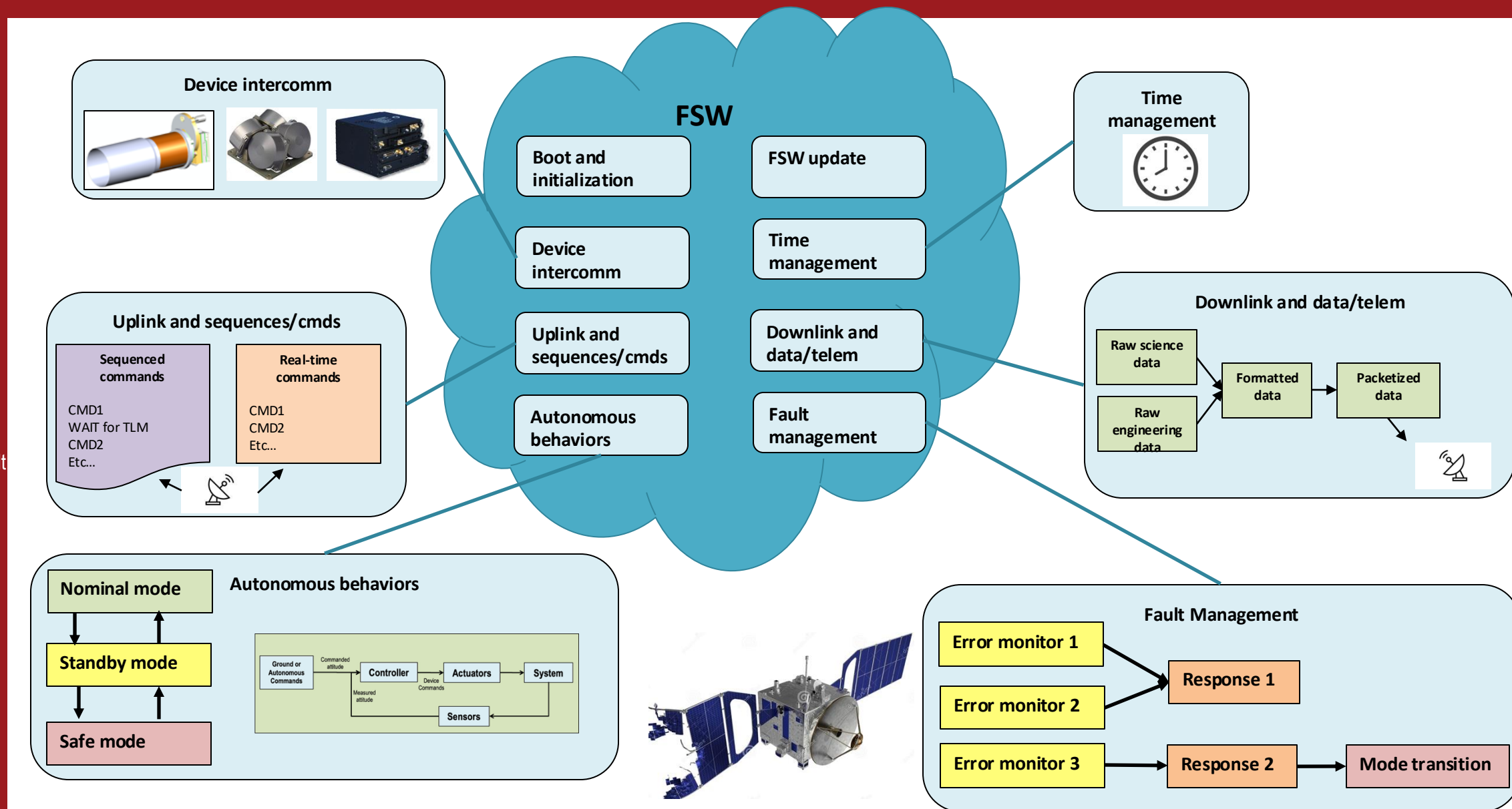
Examples

Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies



Flight Software Architecture Overview

Avionics

Introduction

- Overview
- HW Overview
- **FSW Overview**
- Examples

Functions

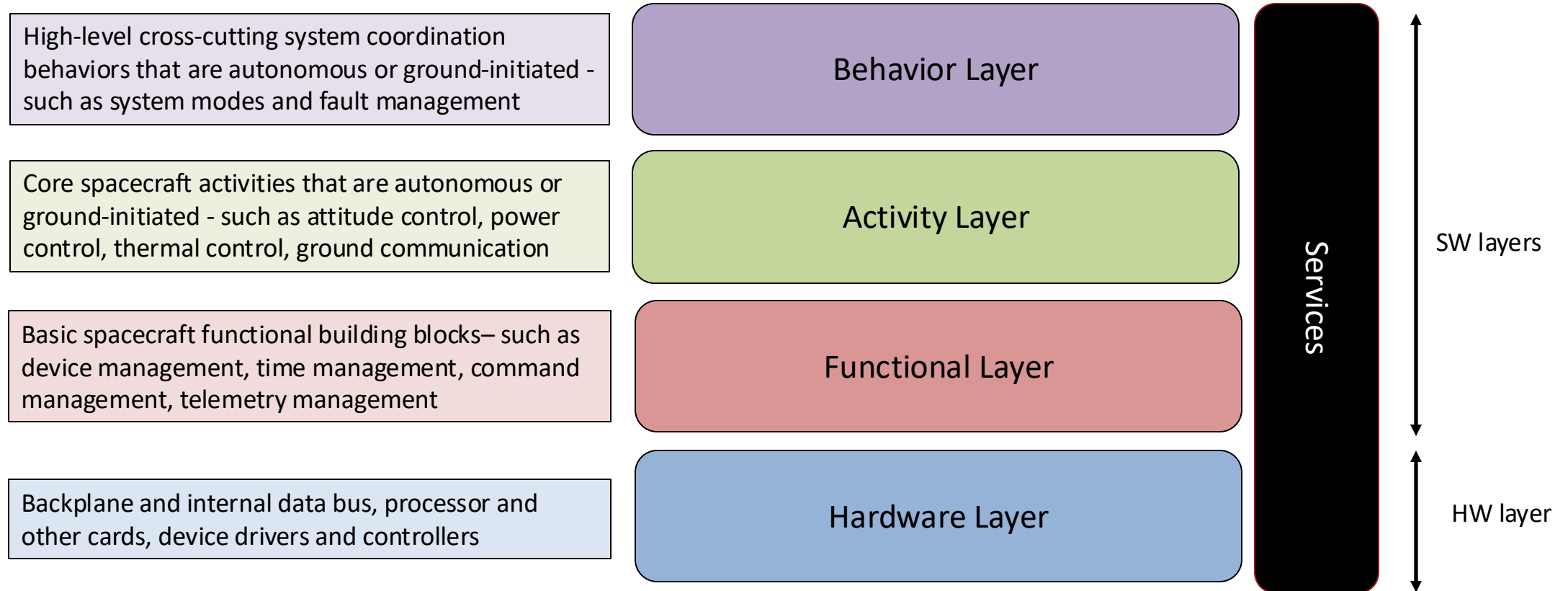
- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

- **FSW architecture is a high-level graphical model (abstraction) of a FSW system**
 - Decomposition of a FSW system into multiple components with interfaces
 - Reflection of FSW requirements (functions, activities, behaviors, interfaces, interactions, etc)
- **System Engineers and FSW Engineers use architecture as the basis for**
 - System function partitioning and allocation
 - Trade studies and design/implementation/V&V decisions making

Key hardware/software services
– such as real time operating system, boot and initialization, FSW upload, memory access



Flight Software Architecture Example

Avionics

Introduction

- Overview
- HW Overview
- **FSW Overview**
- Examples

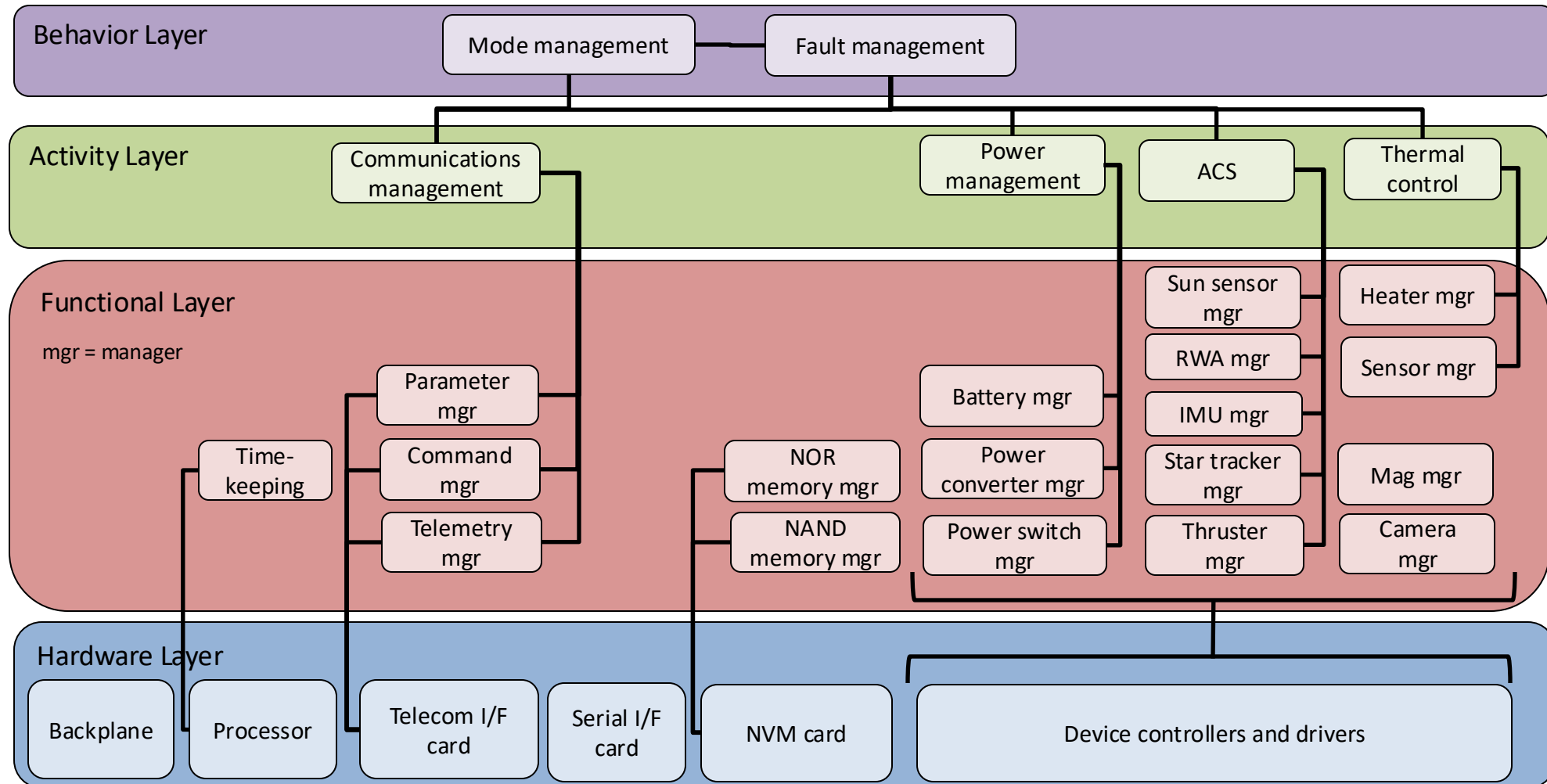
Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

- Within each layer are modules that represent specific functions with the FSW, which are coded by FSW developers and have interfaces to other modules (examples shown below)



Services

Memory access

FSW upload

Boot and init

Real-time operating system

Flight Software Overview

Avionics

Introduction

- Overview
- HW Overview
- **FSW Overview**

Examples

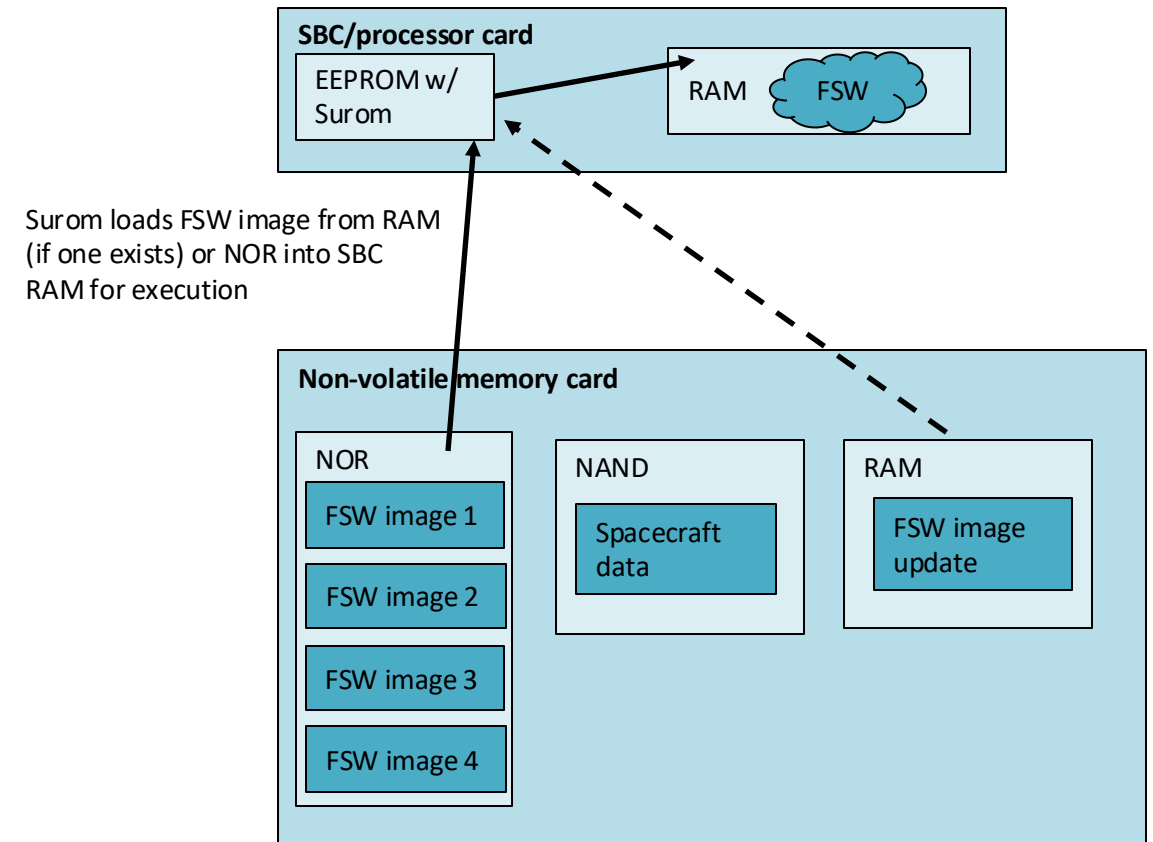
Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

- **Flight Software (FSW)** contains the logic/code necessary to execute the various avionics functions
- **FSW has a boot and initialization function, to ensure that it comes “online” in a reliable and repeatable way**
 - Multiple images of FSW are stored in NOR memory on the NVM card, to ensure robustness to corruption
 - Surom on the SBC selects the FSW image from NOR and loads it into SBC RAM for execution
- **FSW can generally be configured in flight, either through a “patch” or a complete reimage**
 - New FSW image loaded by ground to NVM RAM
 - Surom checks NVM RAM when loading FSW image and loads from there if one exists
- **FSW resides on the flight computer, as well as on peripheral devices such as payloads**



Example: Solar Probe

Avionics

Introduction

- Overview
- HW Overview
- FSW Overview
- **Examples**

Functions

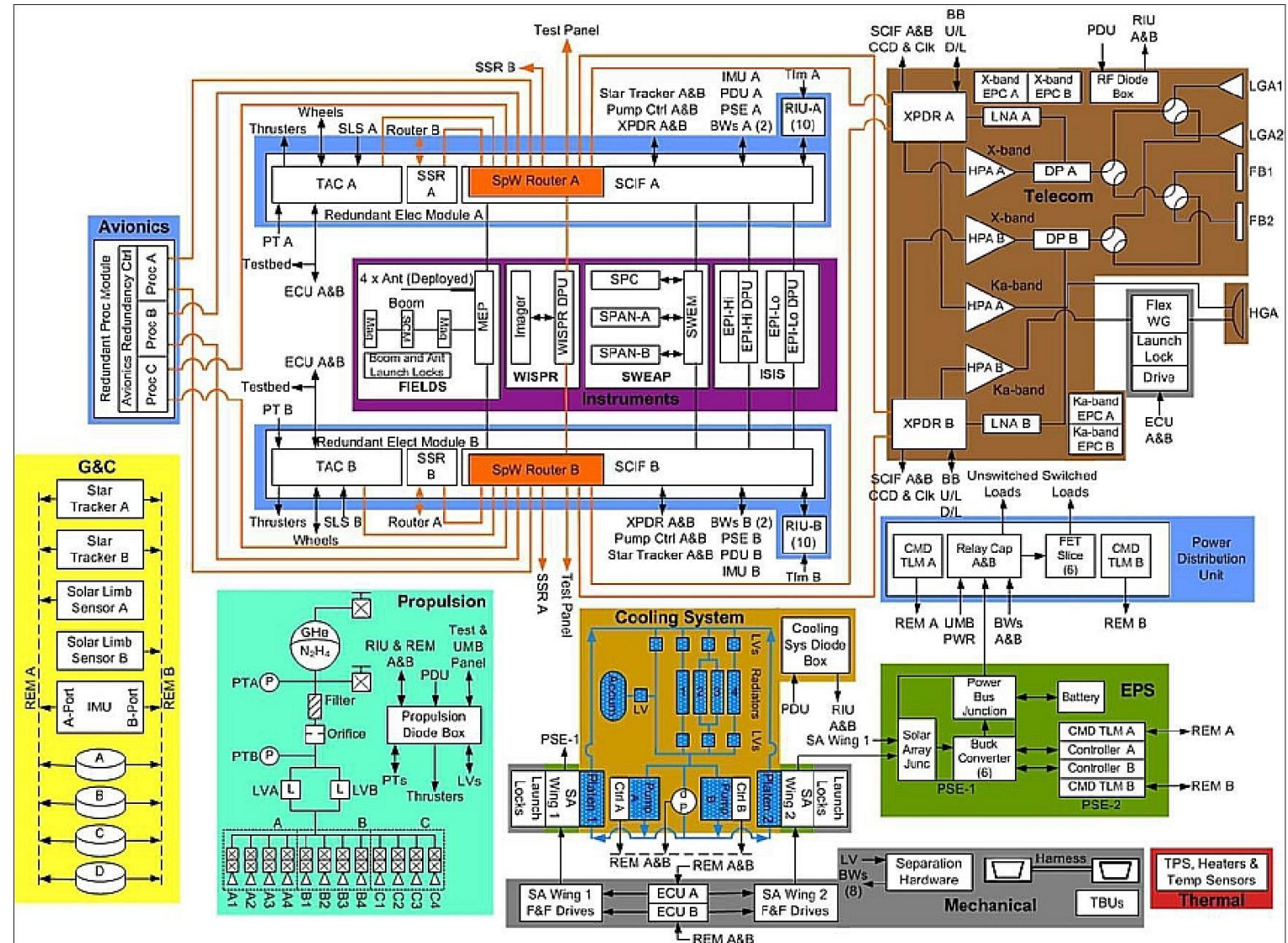
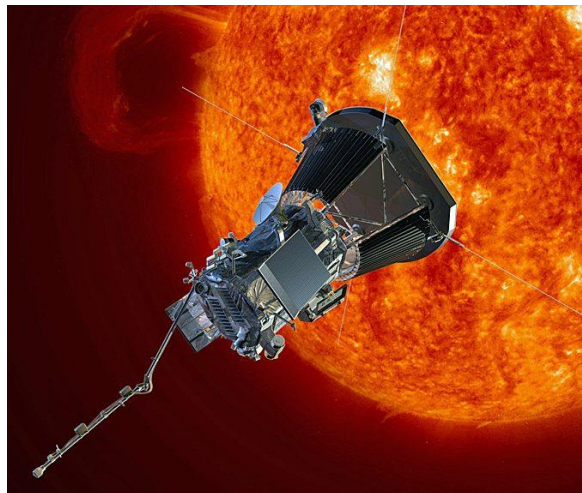
- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

• Key Avionics design drivers:

- Flagship mission – drives overall redundancy across the system
- Tight pointing constraints due to thermal environment - drives very fast recovery time, triple redundant processors
- Several instruments and other spacecraft devices – drivers interface card and memory sizing
- Severe thermal and radiation environment – drives overall chassis design, shielding, and test program



Example: Psyche

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- **Examples**

Functions

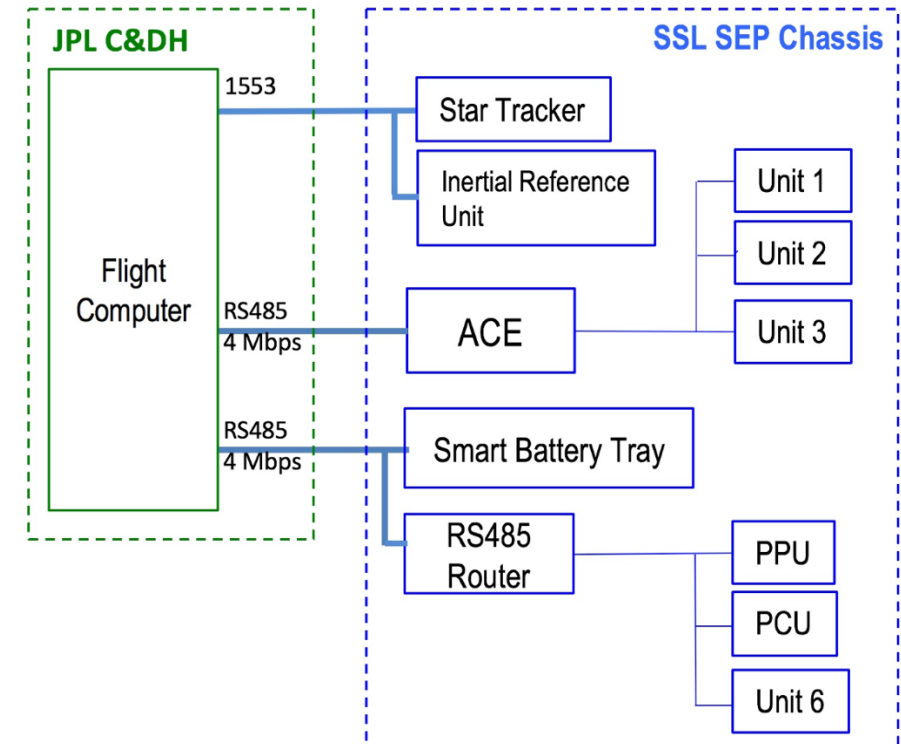
- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

• Key Avionics design drivers:

- Class B mission – drives overall redundancy across the system
- Tight pointing constraints for EP thrusting, science instruments – drives recovery time
- Split avionics design between JPL and Maxar (formerly SSL) – drives use of centralized processor with additional hardware (Attitude Control Electronics [ACE], Router) for communication distributed Maxar avionics hardware
- Maxar avionics heritage – drives new interfaces to JPL hardware, operational strategy (including fault recovery)



Note: redundant units are not shown in this diagram

Avionics, Part 2 – Avionics Functions

Avionics Functions: Processing

Overview

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

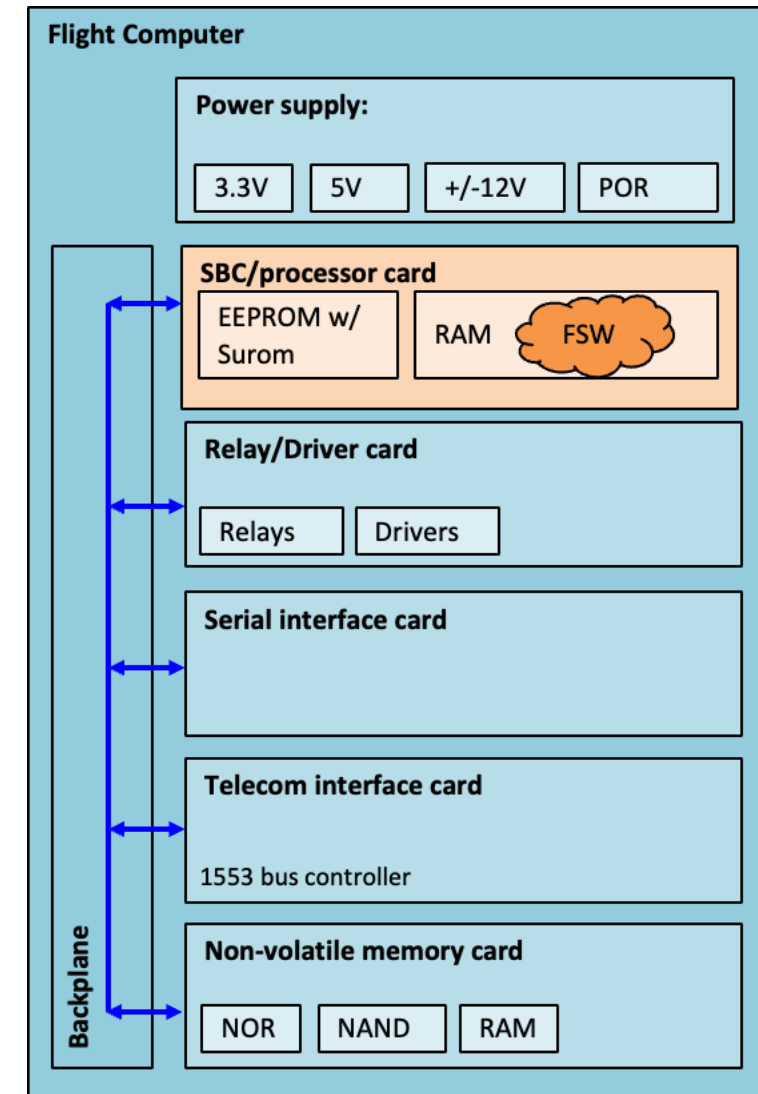
Functions

- **Processing**
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

- **Role:** The processing function is responsible for:
 - Hosting flight software, which provides overall command and control functionality
 - Providing boot and initialization function
- **Hardware:**
 - SBC/processor card
 - Hosts CPU and flight software
 - Provides boot and initialization function
 - Some missions will have additional avionics hardware for specific device interfaces (e.g., Router) – ref: Psyche
- **Software:**
 - Surom – executes boot and initialization
 - Primary flight software – executes most other spacecraft control functions (e.g., attitude control, power management, thermal management, system modes, fault management)
- **Key design trades:**
 - Performance vs reliability, especially in type of flight computer
 - Simplicity vs robustness, especially in redundancy and operational approach (e.g., cold spare vs warm spare)



Avionics Functions: Processing

Types of flight computers/processors

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

Functions

- **Processing**
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

Processor Type	Pros	Cons	Example Hardware	Example Missions
Central Processing Units (CPUs)	Versatile and capable of handling multiple applications and tasks	May not be optimized for real-time operations, can be less fault-tolerant	BAE Systems RAD750	Mars Science Laboratory (Curiosity Rover): Used for scientific data processing and control
Real-Time Processors	Ensures deterministic and timely task execution	May have limited flexibility, can be complex to integrate with other systems	VxWorks-based processors (e.g., Wind River VxWorks 653) RTEMS-based processors (e.g., RAD750, LEON-3)	Juno: Managed critical tasks in Juno's mission around Jupiter
Multi-core Processors	High processing power with parallel task handling, efficient for complex operations.	Can be more power-hungry, increased software complexity for managing multiple cores.	Intel's Space-grade processors (e.g., Intel i7 for space applications)	James Webb Space Telescope (JWST): Processes the vast amount of data collected from IR observations
Integrated Circuit: Application-Specific Integrated Circuits – ASICs	Compact design integrating multiple functions, can be highly efficient for specific tasks	Limited flexibility for changes, custom designs can be costly and time-consuming.	Space Systems/Loral's ASICs Qualcomm Snapdragon	Jupiter Icy Moons Explorer (JUICE): Utilizes integrated systems for efficient operation
Integrated Circuit: Field-Programmable Gate Arrays (FPGAs)	Highly flexible and customizable, supports parallel processing	Complex design and programming, can be costly.	Xilinx Virtex-5/6/7 Series Altera (Intel) Stratix and Arria Series	Hubble Space Telescope: Uses FPGAs for real-time image processing and data handling.

Avionics Functions: Processing

Types of flight computers/processors

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

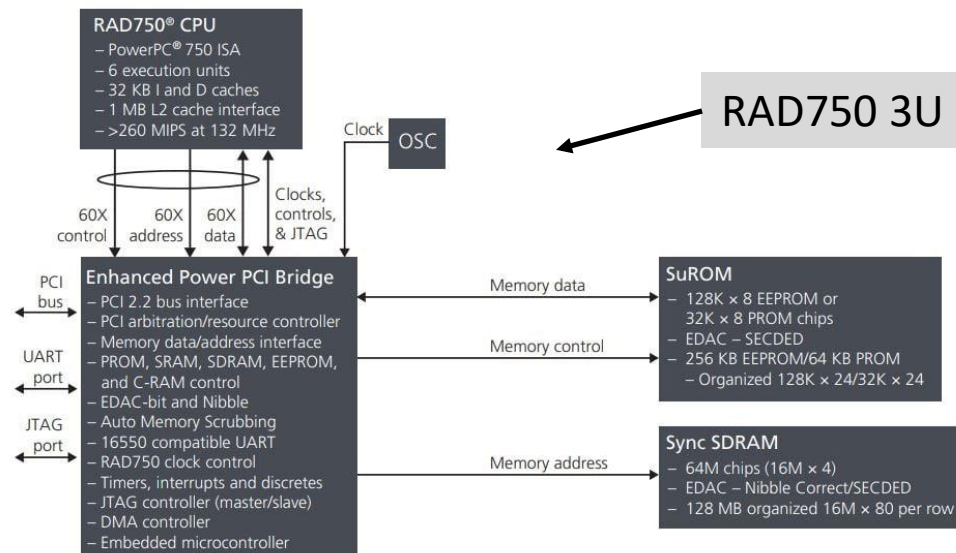
Functions

- **Processing**
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

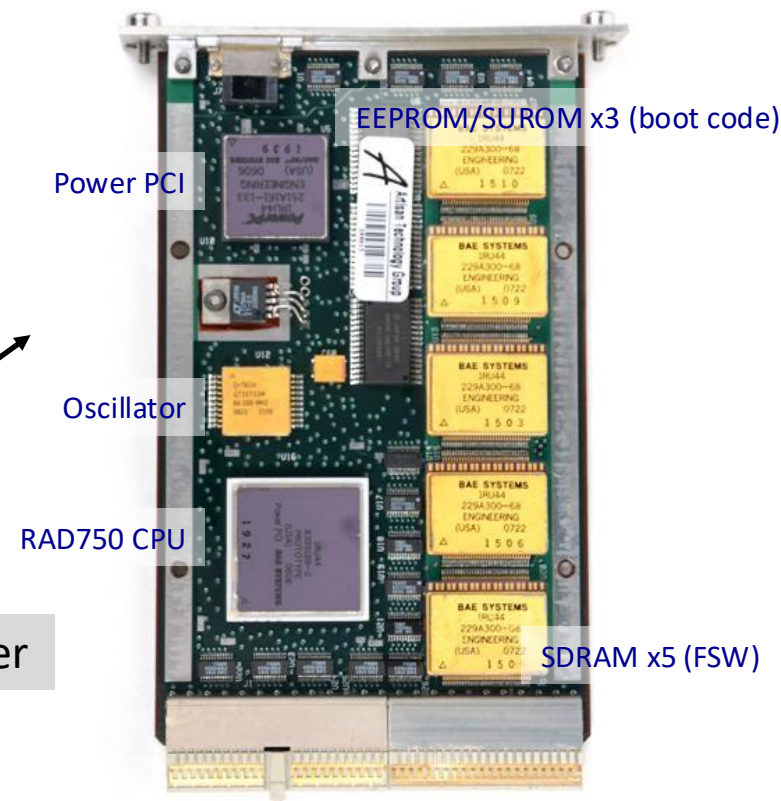
Design Process

Case Studies

- **Processors can be adapted for specific space mission needs, such as:**
 - Radiation hardening - can be costly and reduce performance
 - Fault tolerance - can add complexity
 - Can add block redundancy (e.g., 2x RAD750) or internal redundancy (e.g., triple modular redundancy)
 - Payload processing – limited to Payload-specific operations
 - Most science instruments have their own internal processor



RAD750 3U cPCI Single Board Computer



- **The RAD750 was manufactured by BAE Systems under contract from Lockheed Martin and JPL to support the Deep Impact mission (2005)**
 - It has been the NASA standard for the past two decades
 - Comes in various form factors (e.g., 3U, 6U, custom)
 - Designed to be radiation tolerant

Avionics Functions: Intercommunications

Overview

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

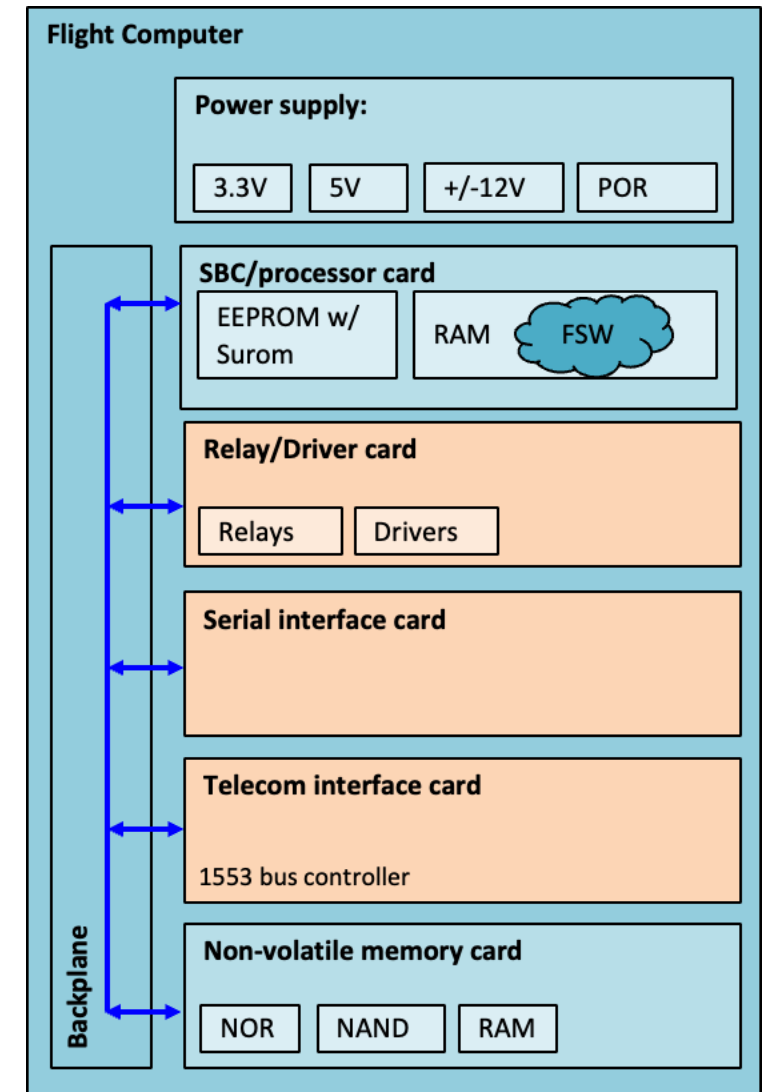
Functions

- Processing
- **Intercomm**
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

- **Role:** The intercommunications function is responsible for:
 - Providing data exchange between all spacecraft subsystems/devices
- **Hardware:**
 - Relay/driver card
 - Provides hardware relays that can be operated independent of FSW (e.g., to reset flight computer, to select FSW image)
 - Provides drivers for propulsion hardware (e.g., latch valves), motor control hardware (e.g., actuators), sensor interfaces (e.g., pressure, temp), etc
 - Serial data interface card
 - Telecom interface card
 - Provides point-to-point interface to spacecraft devices
 - Telecom interface card
 - Provides interface to telecom radio
 - Can provide 1553 bus controller
- **Software:**
 - Data interface initialization
 - Data formatting and constraints
 - Data distribution
- **Key design trades:**
 - Point to point vs data bus architecture
 - Physical layer and protocol selection



Avionics Functions: Intercommunications

Types of data buses

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

Functions

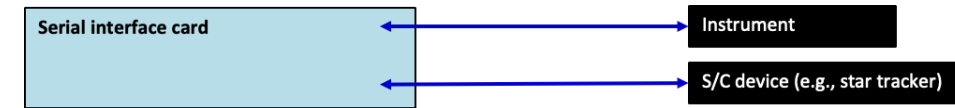
- Processing
- **Intercomm**
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

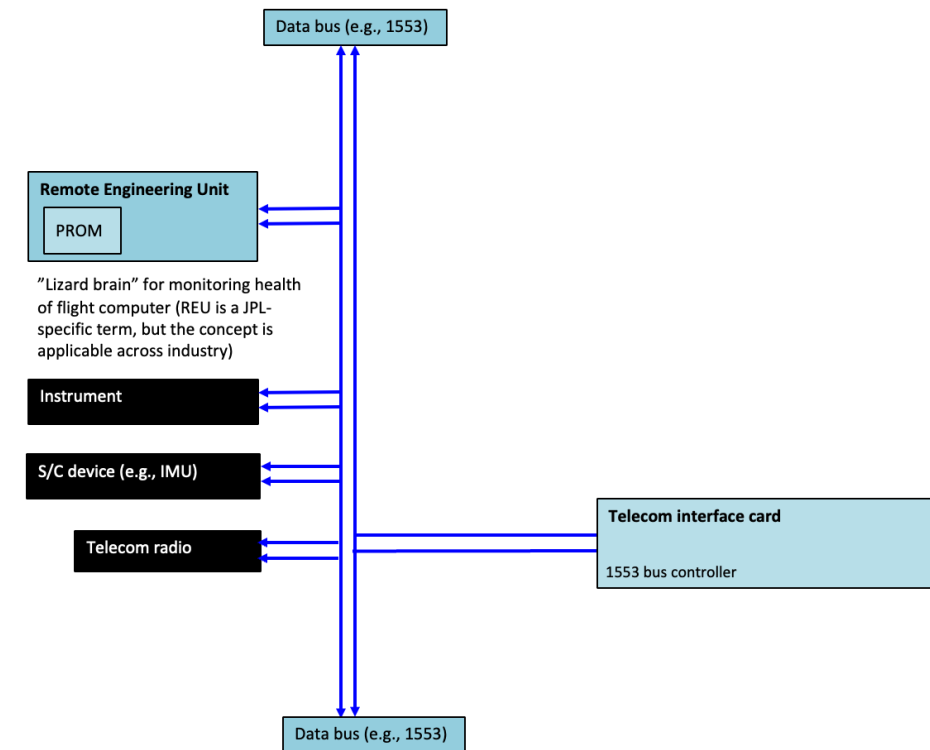
• Point-to-point data interfaces

- Direct and dedicated communication link between two devices
- Use when only a few devices need to communicate directly, and/or a device needs a robust dedicated interface
- Advantages: Simple, non-interfering, can be very high rate
- Disadvantages: Limited scalability, limited flexibility
- Examples: RS-422, RS-485, SpaceWire, Ethernet



• Data bus interfaces

- Network that allows multiple devices (often called remote terminals – RTs) to communicate over a shared interface
- Used in applications where multiple devices need communication; bus controller arbitrates data flow
- Advantages: Scalable, flexible (allows dynamic reconfiguration), often has advanced features (e.g., built in error detection)
- Disadvantages: Complex, devices can interfere with each other (e.g., “babbling”)
- Examples: 1553, CAN, Ethernet



Avionics Functions: Intercommunications

Data Interface Specification

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

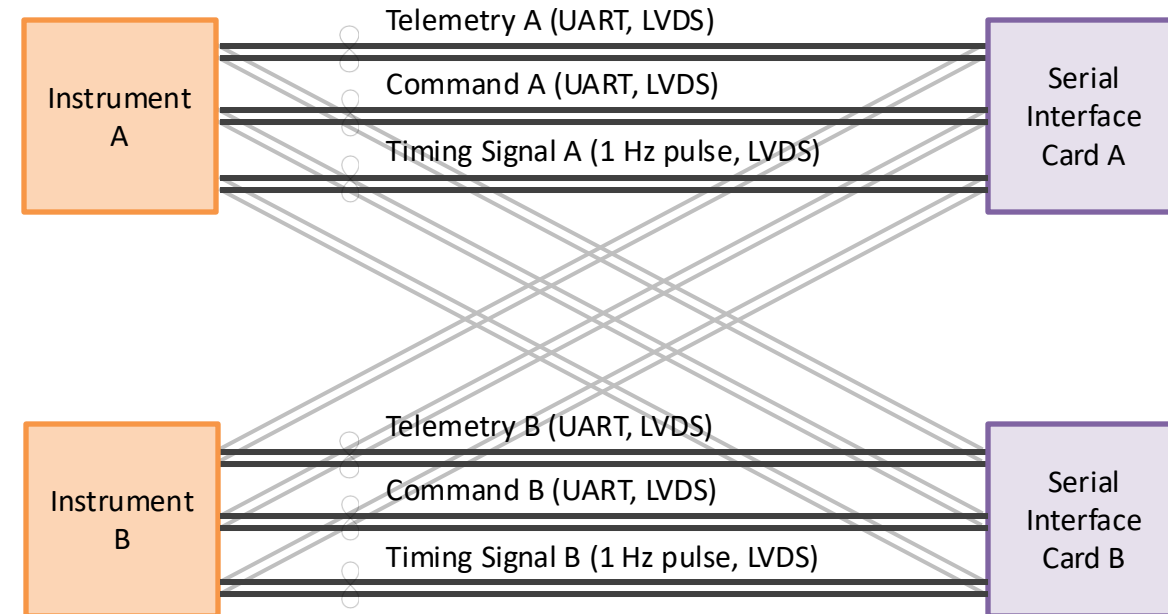
Functions

- Processing
- **Intercomm**
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

- **Physical Layer: Defines the *hardware and electrical specifications* for transmitting data**
 - **Signal Type:** Specifies if data is transmitted as electrical signals, optical signals
 - **Transmission Medium:** Defines the medium used for transmission, such as copper cables, fiber optics, or wireless
 - **Data Rate:** Determines speed at which data is transmitted (e.g., 1 Mbps, 10 Gbps)
 - **Electrical Specifications:** Defines required voltage levels, current limits, and impedance matching requirements
 - **Examples:** RS-422, RS-485, LVDS



- **Protocol Layer: Defines *the structure, format, and management* for communication between devices**
 - **Data Framing:** Specifies how data is organized into frames or packets, including header and footer information
 - **Error Detection and Correction:** Implements methods to detect and correct errors in data transmission
 - **Flow Control:** Manages the rate of data transmission to ensure no loss or overflow
 - **Addressing:** Defines how devices are identified on the network to ensure that data is sent to the correct device
 - **Protocol Specifications:** Includes the set of rules for starting and stopping data flow and managing data exchanges
 - **Examples:** UART, Spacewire

Avionics Functions: Intercommunications

Trade study: Ethernet vs 1553

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

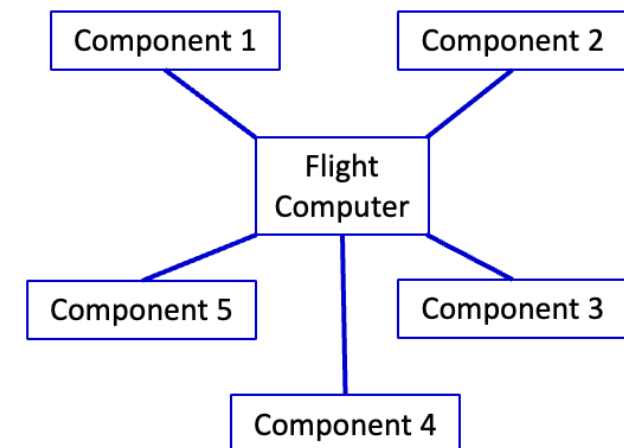
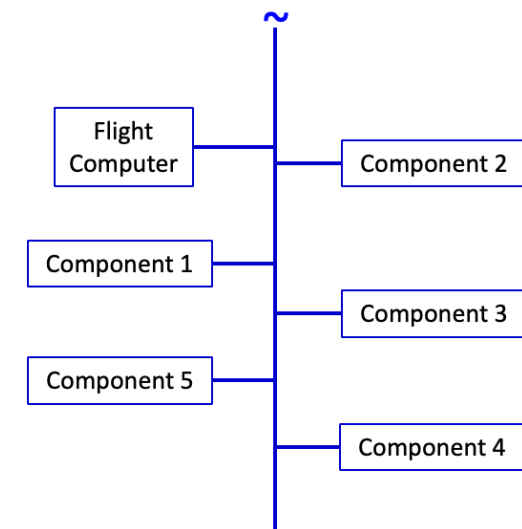
Functions

- Processing
- **Intercomm**
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

- **JPL tends to rely heavily on high heritage serial interfaces or 1553 bus interface, while “new space” companies may rely more on ethernet**
- **1553 – “federated bus” (common multi-drop data bus)**
 - Only up to 1 Mbps
 - Standard interface, deterministic
 - Uses fixed format messages for data transfer
 - Limited scalability
 - Highly reliable, with built in error correction
- **Ethernet – “star bus” (discrete interfaces between each system with a central processor managing all data channels)**
 - Faster data rates, up to 10s of Gbps
 - Custom interface for each device
 - Uses flexible frames/packets (smaller segments of data) for data transfer
 - Highly scalable
 - May be susceptible to network collisions



Avionics Functions: Uplink and Command Management

Overview

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

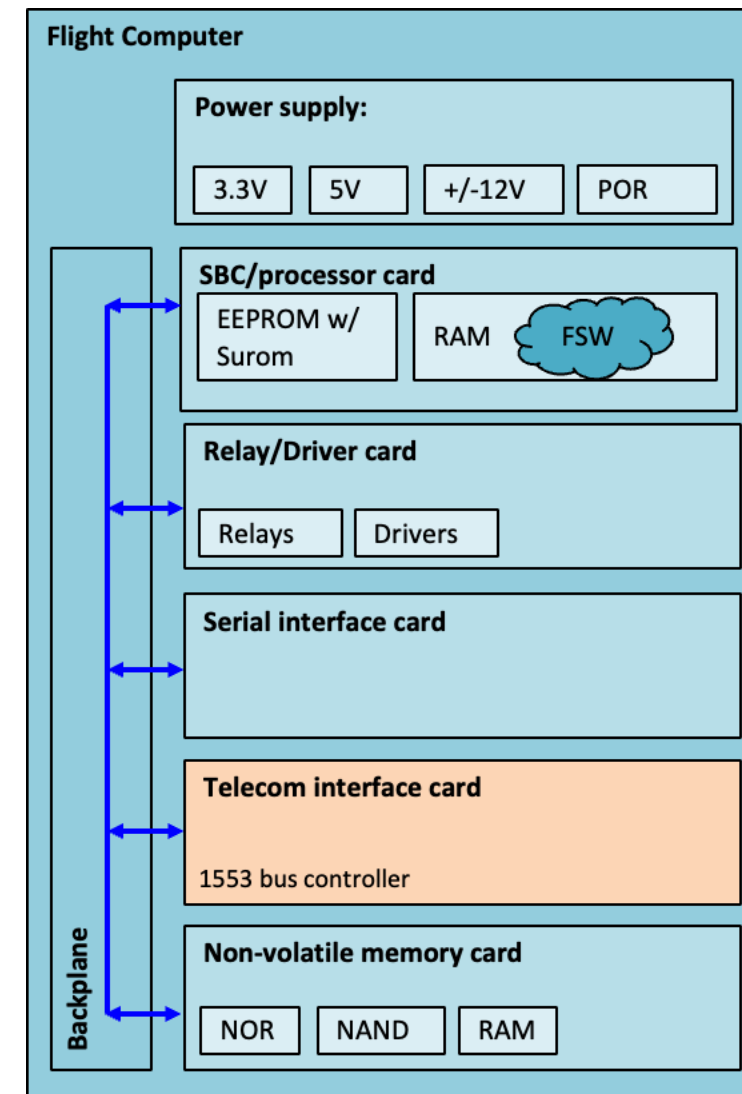
Functions

- Processing
- Intercomm
- **Uplink/command**
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

- **Role:** The uplink and command management function is responsible for:
 - Providing communication from the ground operators to the spacecraft
 - Validating uplink products (real-time commands and files/sequences)
 - Decoding commands (i.e., interpreting and executing commands)
- **Hardware:**
 - Telecom interface card
 - Provides interface to telecom radio
 - Can provide 1553 bus controller
- **Software:**
 - Command validation, decoding, execution
 - File/sequence validation, storage
- **Key design trades:**
 - Software commands vs hardware commands
 - Real-time commands vs sequenced commands



Avionics Functions: Uplink and Command Management

Types of commands

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

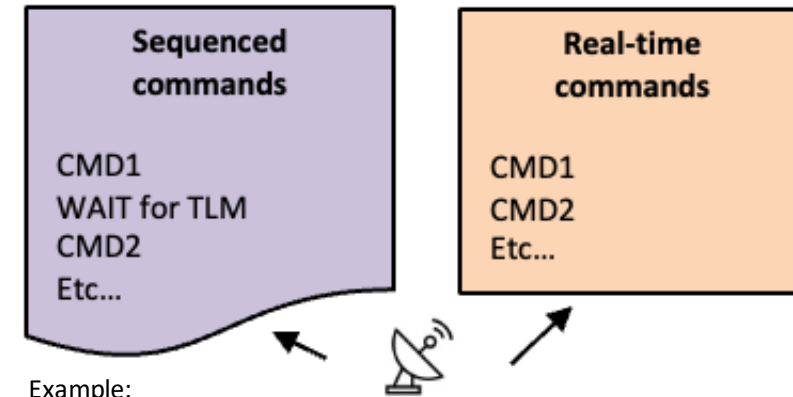
Functions

- Processing
- Intercomm
- **Uplink/command**
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt
- Fault mgmt

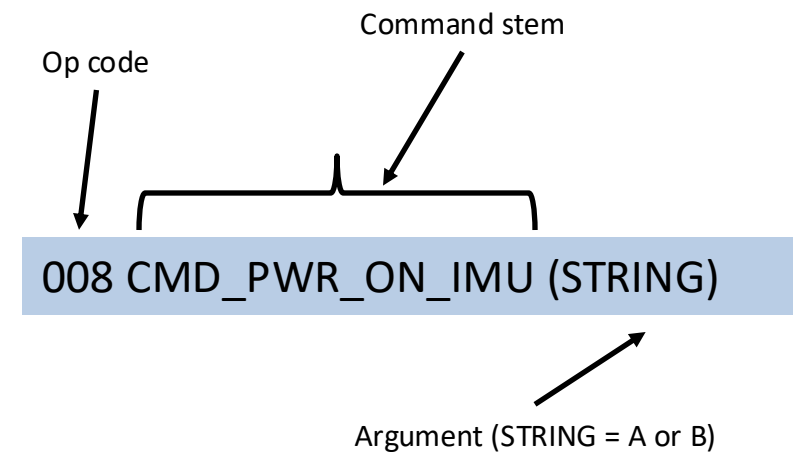
Design Process

Case Studies

- **Commands are the instructions that are sent to the spacecraft that it processes and responds to**
 - **Software commands (most commands):** these are commands that are interpreted and executed by FSW
 - e.g., CMD_PWR_ON_IMU that powers on the IMU
 - Real-time: Individual commands sent with an operator in the loop
 - Sequenced: A file of commands sent that can execute at specific times and with logic/conditional checks, without an operator in the loop
 - **Hardware commands (less common):** these commands that bypass FSW and can be executed directly by the hardware
 - eg, HDW_SYS_RESET that resets FSW
 - Used for fault scenario when FSW may be hung
 - **Formatting**
 - The actual commands sent to the spacecraft are in binary, where each bit has a specific meaning that is defined by the command & telemetry dictionary (C&TD)
 - The C&TD typically specifies the following:
 - Command Stem: The mnemonic associated with the command
 - Op Code: The ID # that is sent to the s/c
 - Arguments: The variables that are inputs to the command
 - Restricted modes: The system modes that disallow that command (e.g., safe mode may disallow instrument power on commands)
 - FSW module: The FSW module that contains the command code
 - Descriptions, etc



Example:
 CMD_PWR_ON_IMAGER
 WAIT until TLM_IMAGER_STATE = ON
 CMD_COLLECT_IMAGE
 WAIT 300s
 CMD_PWR_OFF_IMAGER



Avionics Functions: Downlink and Data Management

Overview

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

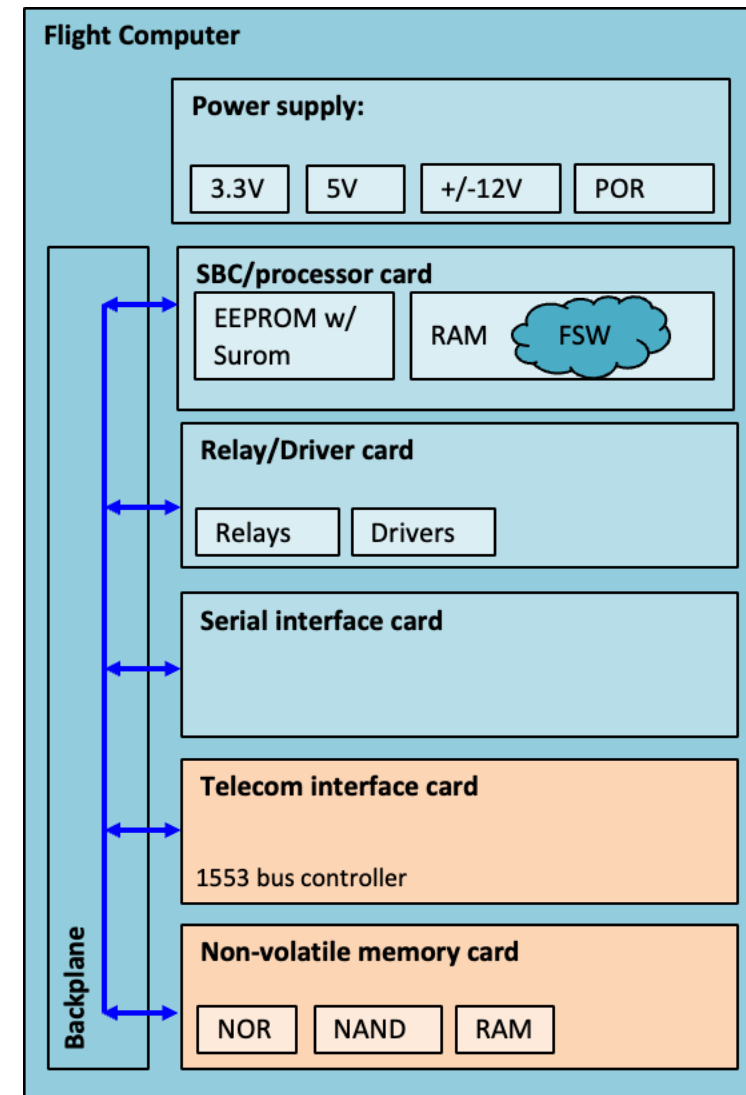
Functions

- Processing
- Intercomm
- Uplink/command
- **Downlink/data**
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

- **Role:** The downlink and data management function is responsible for:
 - Collecting data from all subsystem devices
 - Processing data for onboard use
 - Storing data for future downlink to the ground
 - Packetizing data for downlink to the ground
- **Hardware:**
 - Non-volatile memory card
 - Provide bulk data storage for housekeeping data and payload data
 - Provides data storage of redundant FSW images
 - Telecom interface card
 - Provides interface to telecom radio
- **Software:**
 - Telemetry generation and collection
 - Data storage, formatting, and transmission
- **Key design trades:**
 - Volatile vs non-volatile memory
 - NAND vs NOR memory
 - Parameter management



Avionics Functions: Downlink and Data Management

Types of data/telemetry

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

Functions

- Processing
- Intercomm
- Uplink/command
- **Downlink/data**
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies

- **Engineering, Housekeeping, & Accountability (EHA)**
 - This is channelized telemetry that is provided at regular intervals for all spacecraft devices (instruments, subsystem hardware, etc)
 - Format: [subsystem]-[ID]: Description
 - Example-1: PWR_IMU_A_STATE = [ON/OFF]
This provides the power state of IMU A
 - While the onboard EHA status is continually updated, they are only downlinked at regular intervals (eg, every 5-30 sec)
 - EHA is generally categorized as "housekeeping" or "mission" data
 - Housekeeping data – General health and status of subsystems, used for onboard fault management, ground trending, etc
 - Mission data – The science data collected by the payload – e.g., imagery data, spectrometer data, etc
- **Event Records (EVRs)**
 - These are event records (similar to logs or text messages or tweets) that record specific events at specific times
 - These are NOT common in the industry, as many organizations will instead rely on EHA to provide this information
 - **Formatting**
 - These will include the SC time and both hard-coded and variable responses
 - Format: Time, Event Description
 - Example-1 0000145, Command successfully executed: CMD_CDH_NOOP
 - Example-2: 0000147, Command successfully executed: CMD_CDH_EHA_RATE
 - Example-3: 0000148, EHA Rate set to 5 seconds
 - As with commands, these are transmitted as individual bits/codes, which are then looked up on the ground to translate into a readable description

Avionics Functions: Downlink and Data Management

Types of memory

Avionics

Introduction

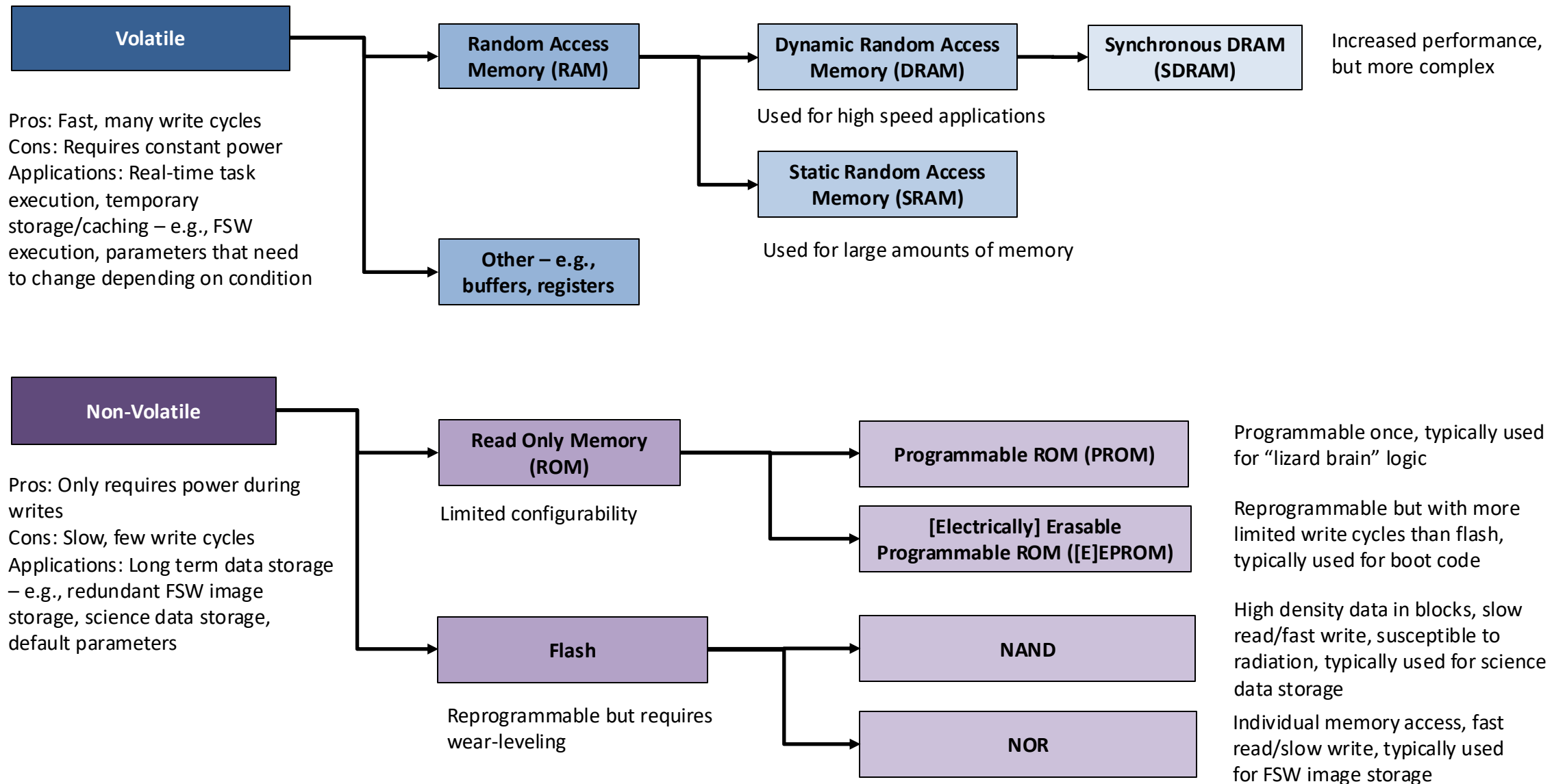
- Overview
- HW Overview
- SW Overview
- Examples

Functions

- Processing
- Intercomm
- Uplink/command
- **Downlink/data**
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies



Avionics Functions: Time Management

Overview

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

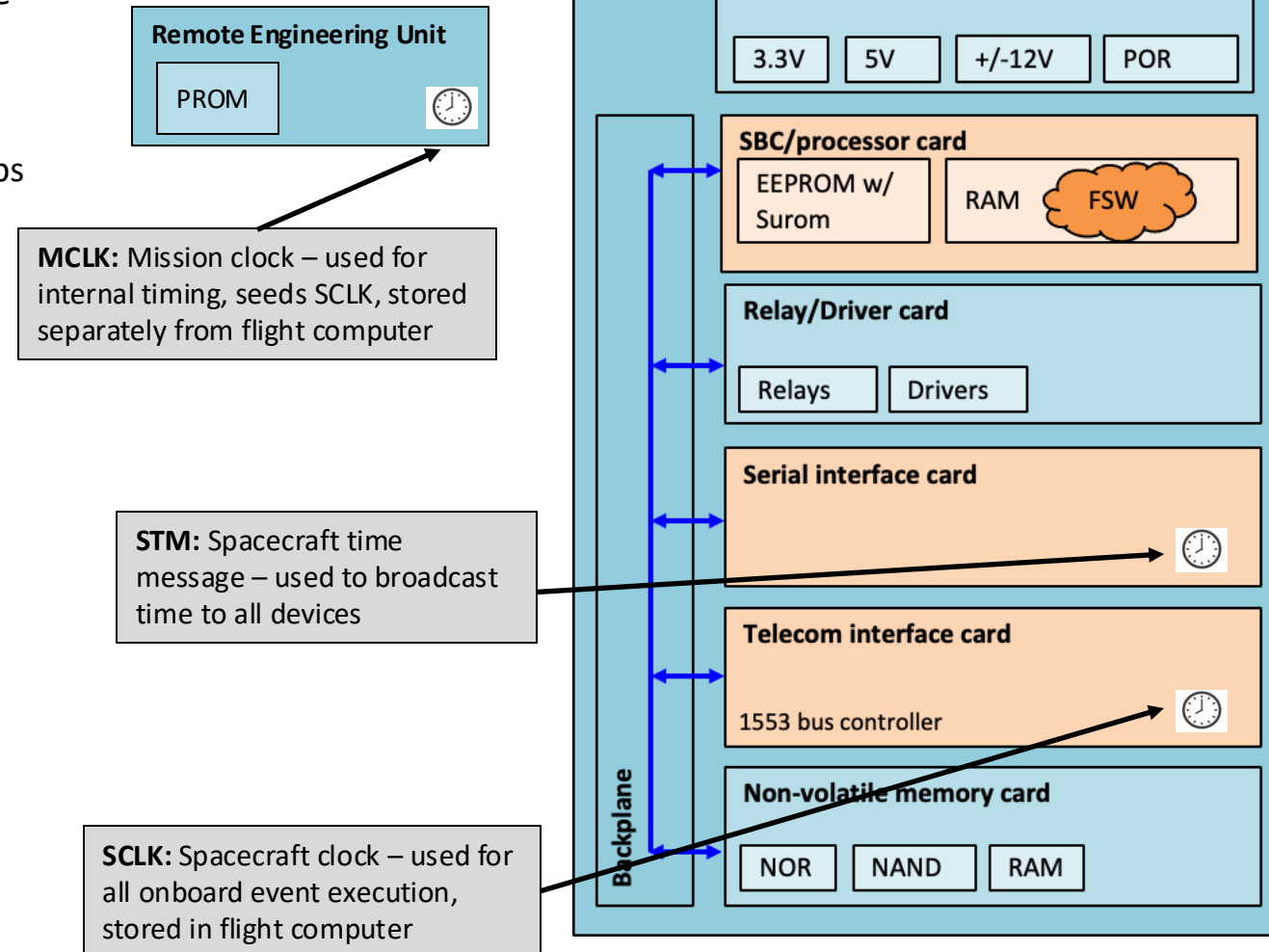
Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- **Time mgmt.**
- Autonomy & Fault mgmt

Design Process

Case Studies

- **Role:** The time management function is responsible for:
 - Managing time within FSW and across the spacecraft devices
 - Correlating spacecraft clocks with ground time
- **Hardware:**
 - Processor card
 - Provides time correlation, monitors for time jumps
 - Telecom interface card
 - Typically stores SCLK
 - Serial interface card
 - Broadcasts STM
 - Remote engineering unit
 - Typically stores MCLK
- **Software:**
 - Seed SCLK with MCLK if FSW resets
 - Distribute STM to devices
- **Key design trades:**
 - Clock precision and stability needs for devices
 - Time broadcast strategy



Avionics Functions: Autonomy and Fault Management

Overview

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

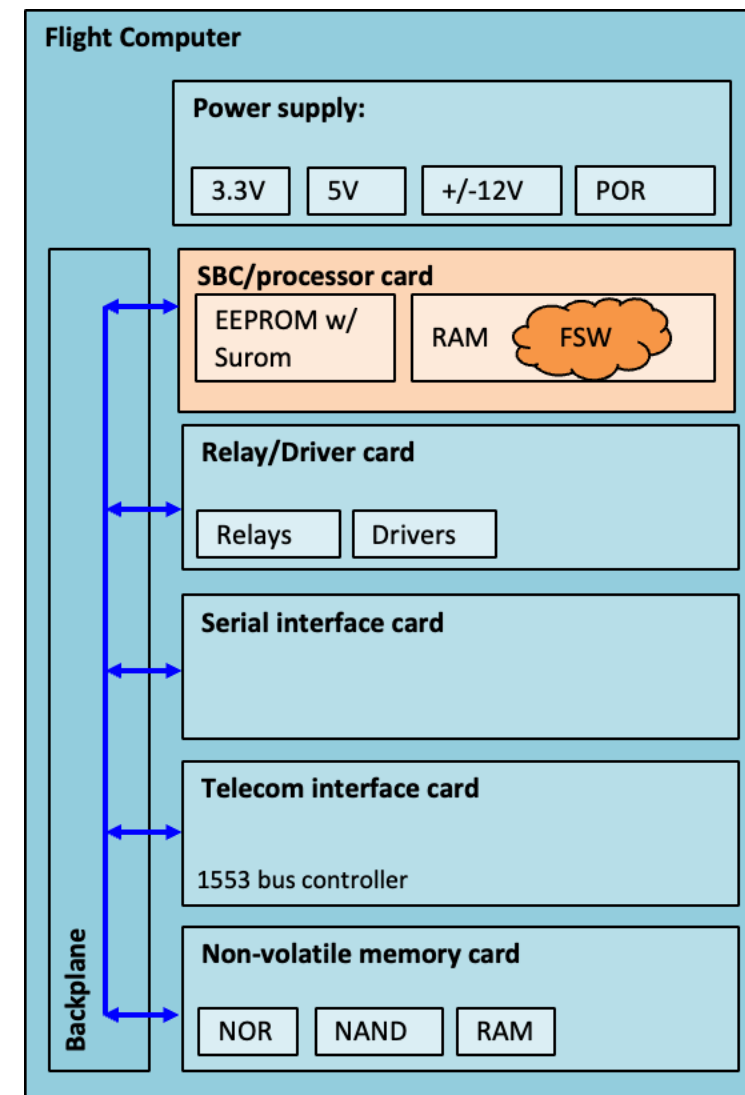
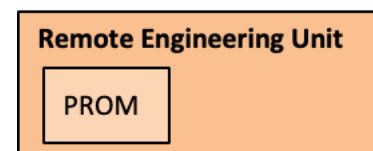
Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- **Autonomy & Fault mgmt**

Design Process

Case Studies

- **Role:** The autonomy and fault management functions are responsible for:
 - Executing on-board FSW behaviors independent ground-operator control, for nominal and off-nominal operation of the spacecraft
- **Hardware:**
 - Processor card
 - Hosts CPU and flight software
 - Remote Engineering Unit (REU)
 - Provides independent health checking and redundancy management for flight computer → if REU detects loss of signal from computer, swaps computer
- **Software:**
 - Attitude determination and control
 - Communications management
 - Power management
 - Thermal management
 - System modes
 - Fault management
- **Key design trades:**
 - Real-time commanding vs sequencing vs onboard autonomy/behaviors



Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

Functions

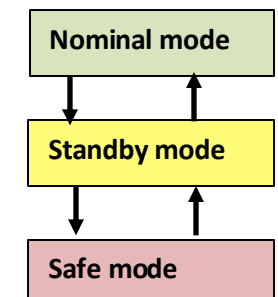
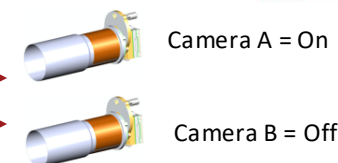
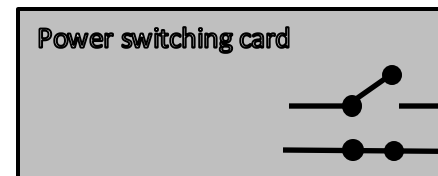
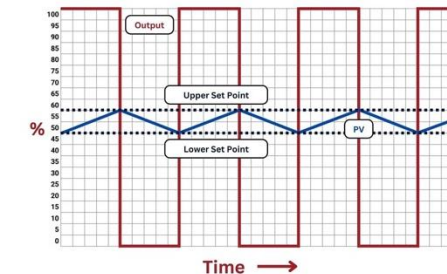
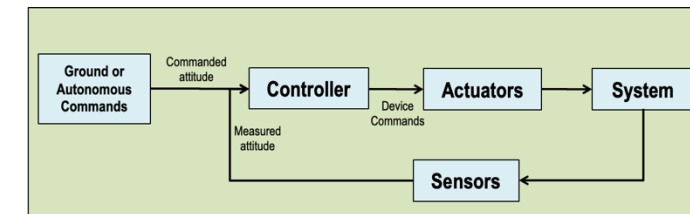
- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- **Autonomy & Fault mgmt**

Design Process

Case Studies

• Autonomous behaviors

- Communications management
 - Executes pre-planned communications windows, including turning to Earth and configuring radio
- Attitude determination and control
 - Determines spacecraft attitude
 - Executes turns, delta-v maneuvers, attitude maintenance, momentum management
- Thermal management
 - Monitors device temperatures and turns heaters on and off
- Power management
 - Turns devices on and off
 - Charges the battery
- System modes
 - Sets spacecraft states for nominal and faulted operations



Avionics Functions: Fault Management

Autonomy capabilities

Avionics

Introduction

- Overview
- HW Overview
- SW Overview
- Examples

Functions

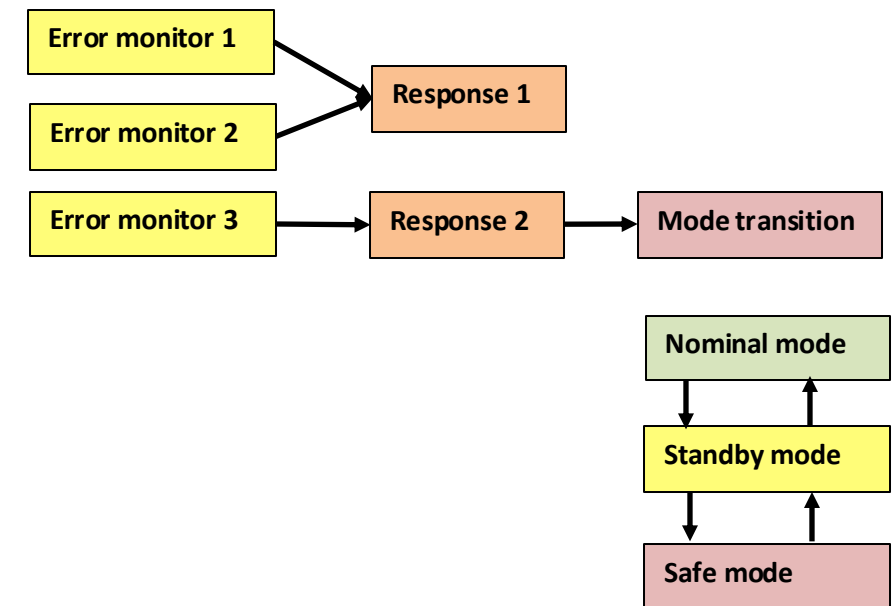
- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- **Autonomy & Fault mgmt**

Design Process

Case Studies

• Fault management

- Error monitors look for off-nominal situations
 - Devices out of temperature limits (too hot, too cold)
 - Battery state of charge too low
 - Attitude control errors too high
 - Etc...
 - Includes condition and parameterized threshold and persistence
 - Condition typically requires FSW update to change
 - Threshold and persistence are typically a parameter that can be changed by ground command
- Response actions correct the problem
 - Power off faulty device
 - Swap to redundant device
 - Transition to safe mode
 - All non-critical devices off (e.g., science instruments)
 - Spacecraft in most basic ACS state – typically IMU and Sun sensors only
 - Spacecraft on low gain antenna communications – high beamwidth for robust communications with Earth
 - Spacecraft in safe attitude – typically solar arrays pointed at Sun
 - Etc...
 - Includes series of actions
 - Response actions and order typically require FSW update to change
- Need to show that spacecraft can remain safe in the presence of faults (e.g., during computer faults/swaps - loss of attitude control, loss of thermal control)



Error monitor: IF [condition] [$>$, $<$, $=$] [threshold] for [persistence]

- IF TLM_PRIME_IMU_TEMP $>$ 30 for 10 SECONDS

Response: THEN [actions]

- CMD_IMU_PRIME [OFF]
- CMD_IMU_BACKUP [ON]
- CMD_SAFE_MODE

Avionics, Part 3 – Design Process

Avionics Design Process

Overview

Avionics

Introduction

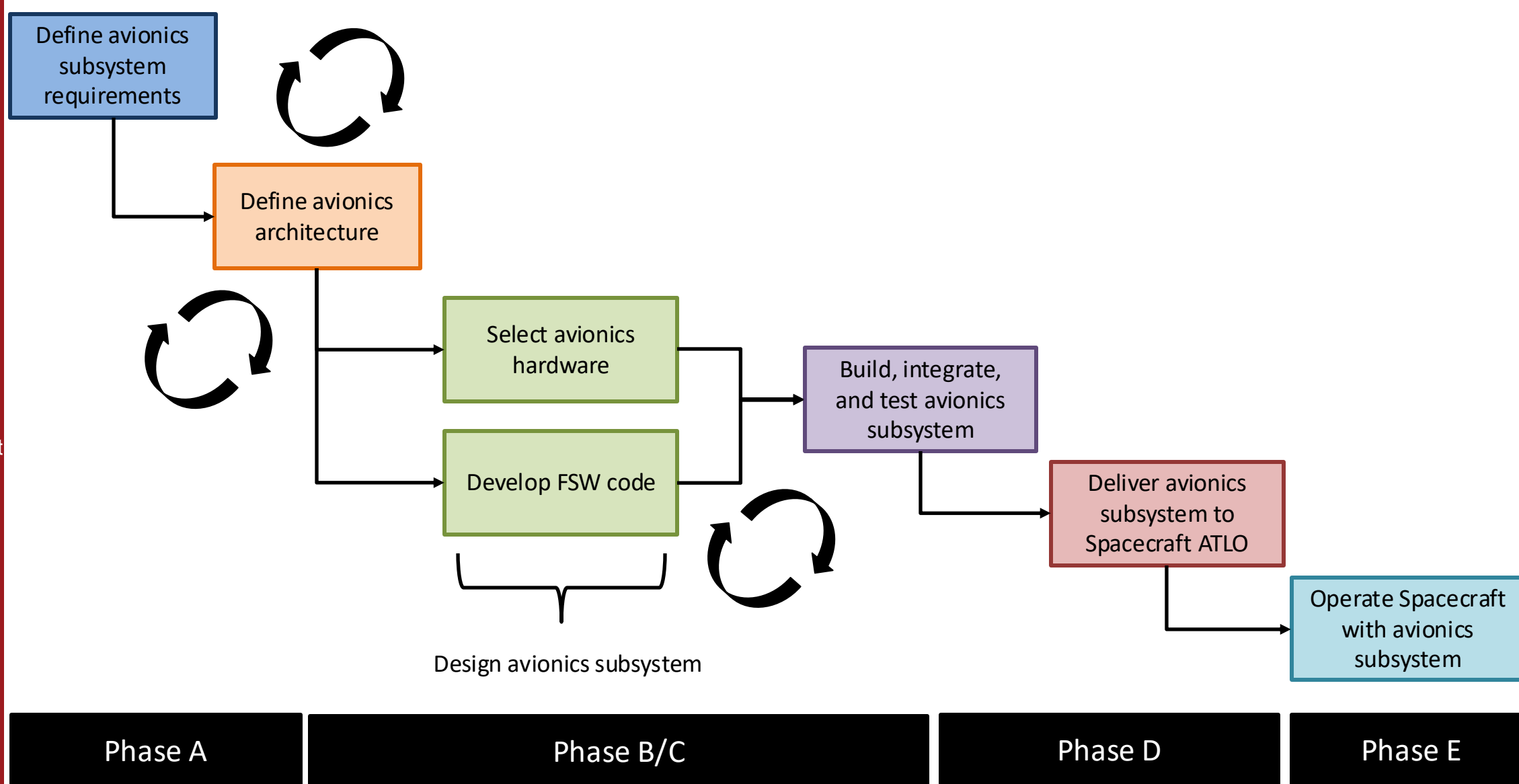
- Overview
- HW Overview
- SW Overview
- Examples

Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies



Avionics Design Process

Requirements

Avionics

Introduction

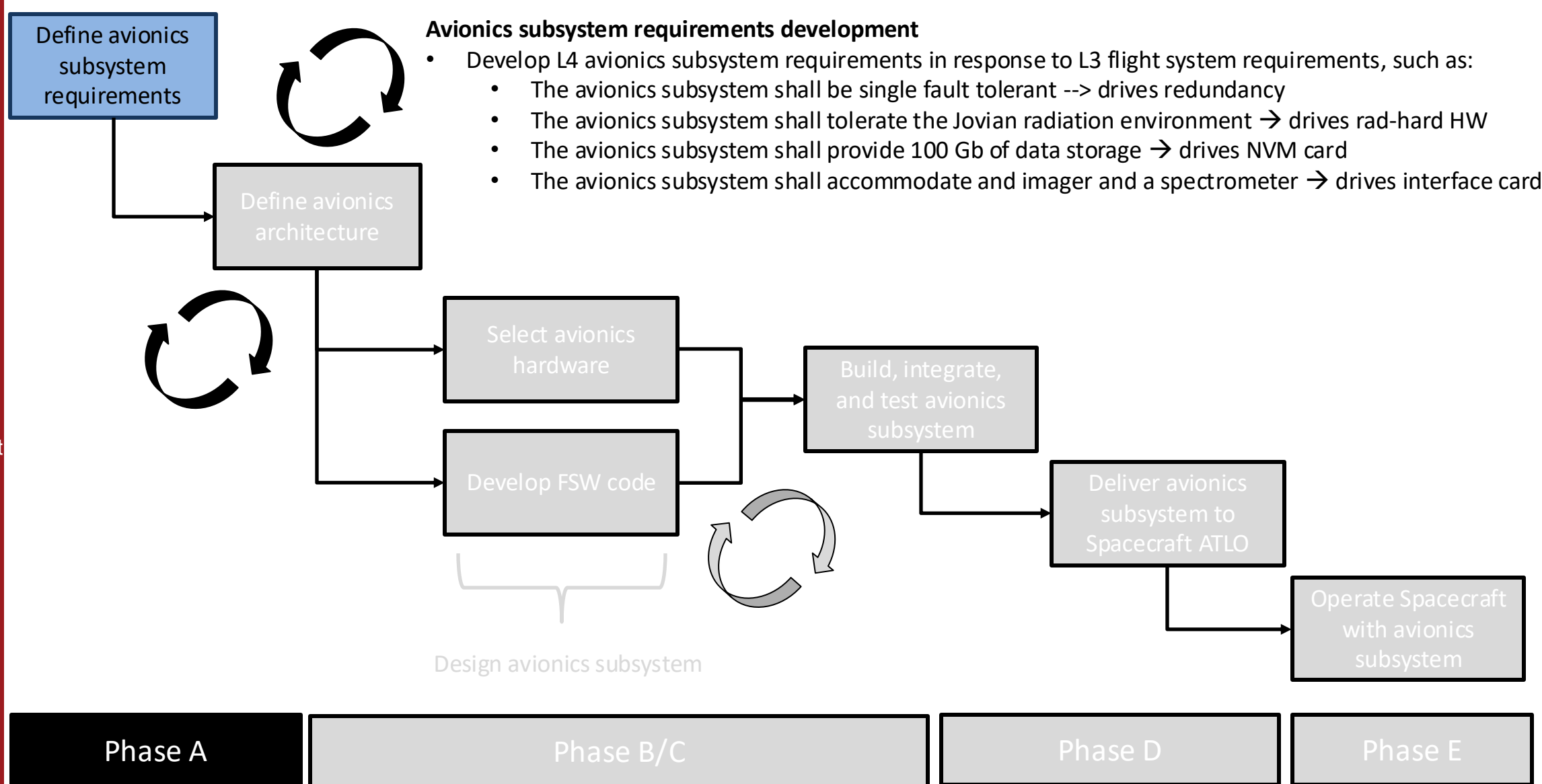
- Overview
- HW Overview
- SW Overview
- Examples

Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies



Avionics Design Process

Architecture

Avionics

Introduction

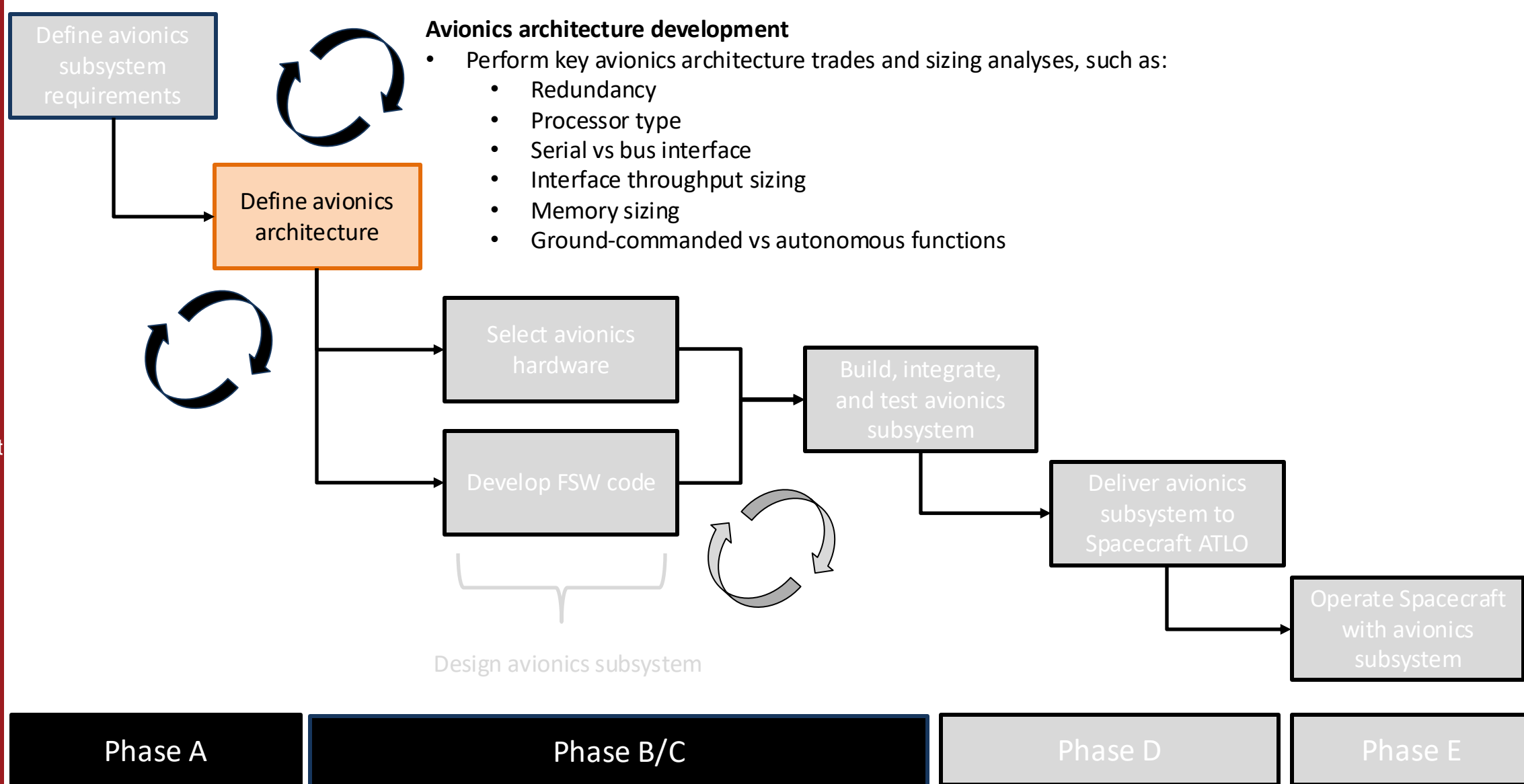
- Overview
- HW Overview
- SW Overview
- Examples

Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies



Avionics Design Process

Design

Avionics

Introduction

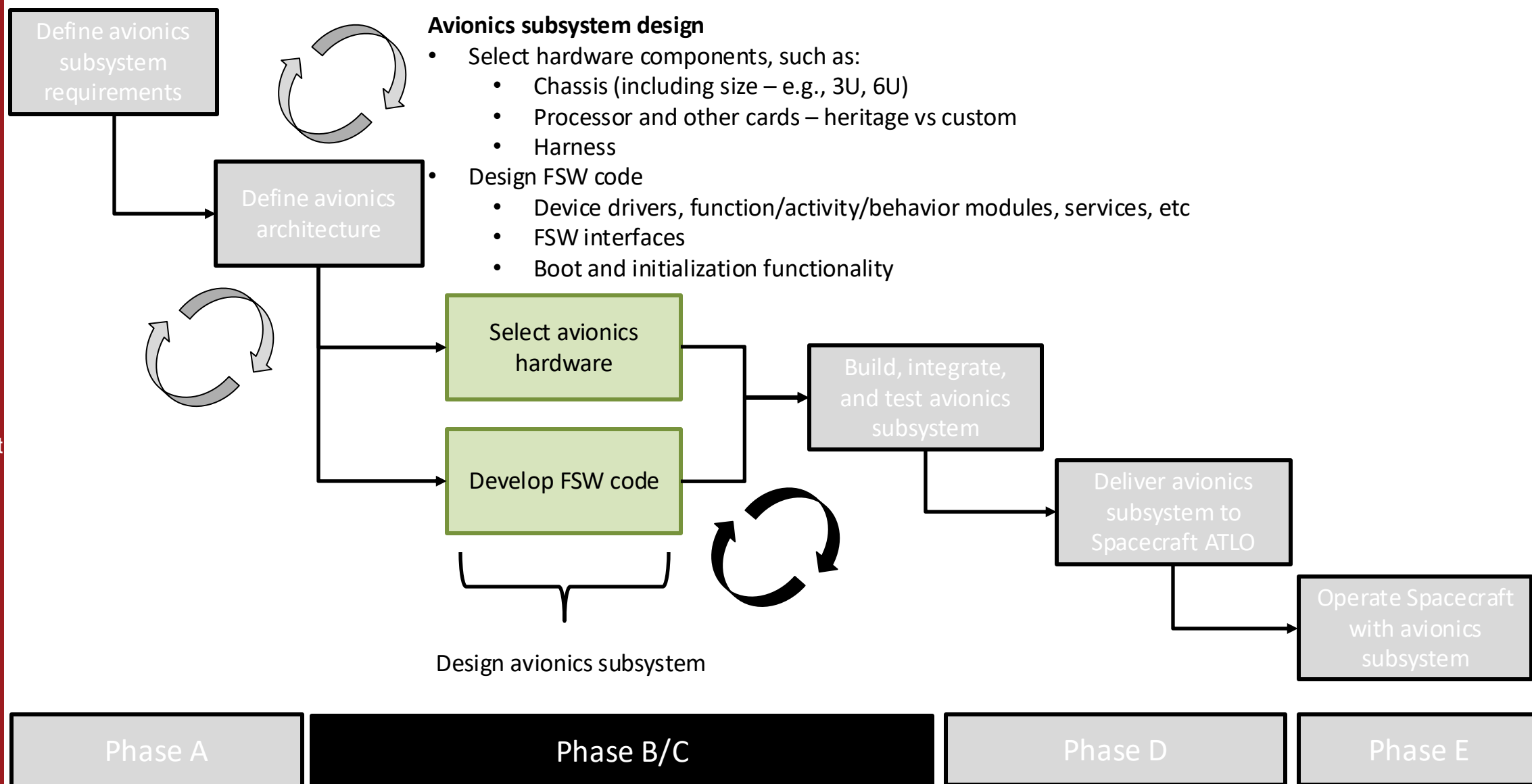
- Overview
- HW Overview
- SW Overview
- Examples

Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies



Avionics Design Process

Build and Test

Avionics

Introduction

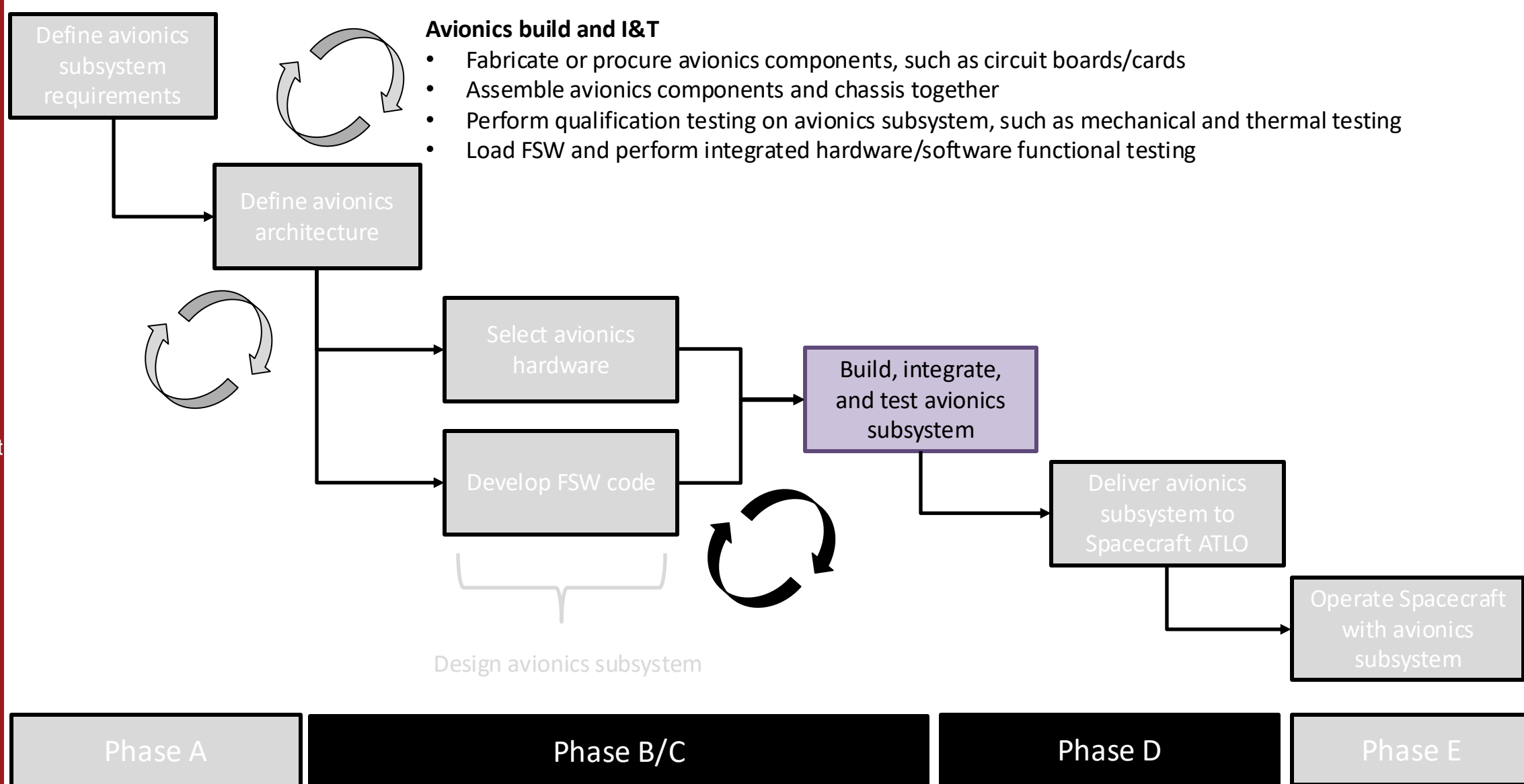
- Overview
- HW Overview
- SW Overview
- Examples

Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies



Avionics Design Process

Deliver

Avionics

Introduction

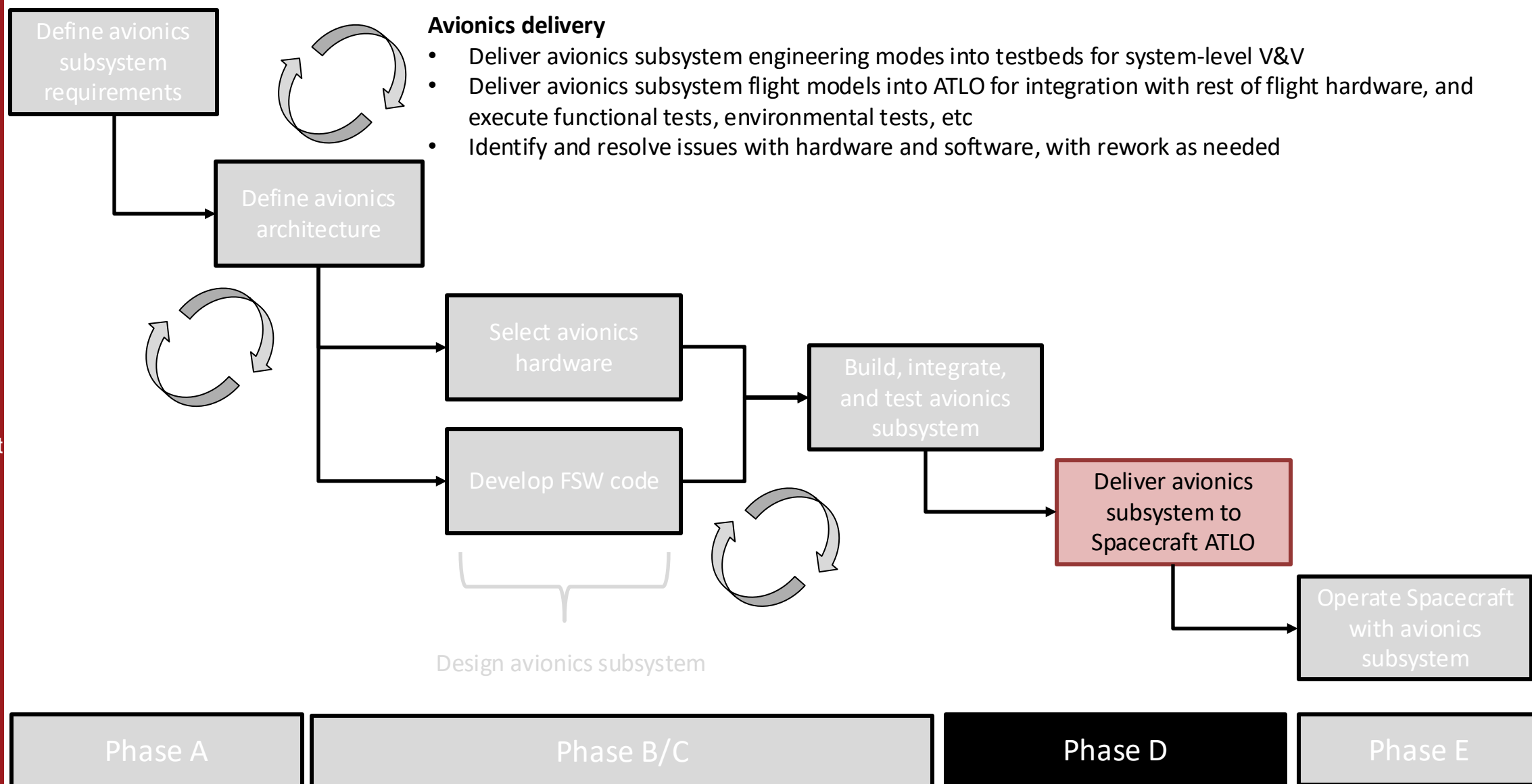
- Overview
- HW Overview
- SW Overview
- Examples

Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

Design Process

Case Studies



Avionics Design Process

Operate

Avionics

Introduction

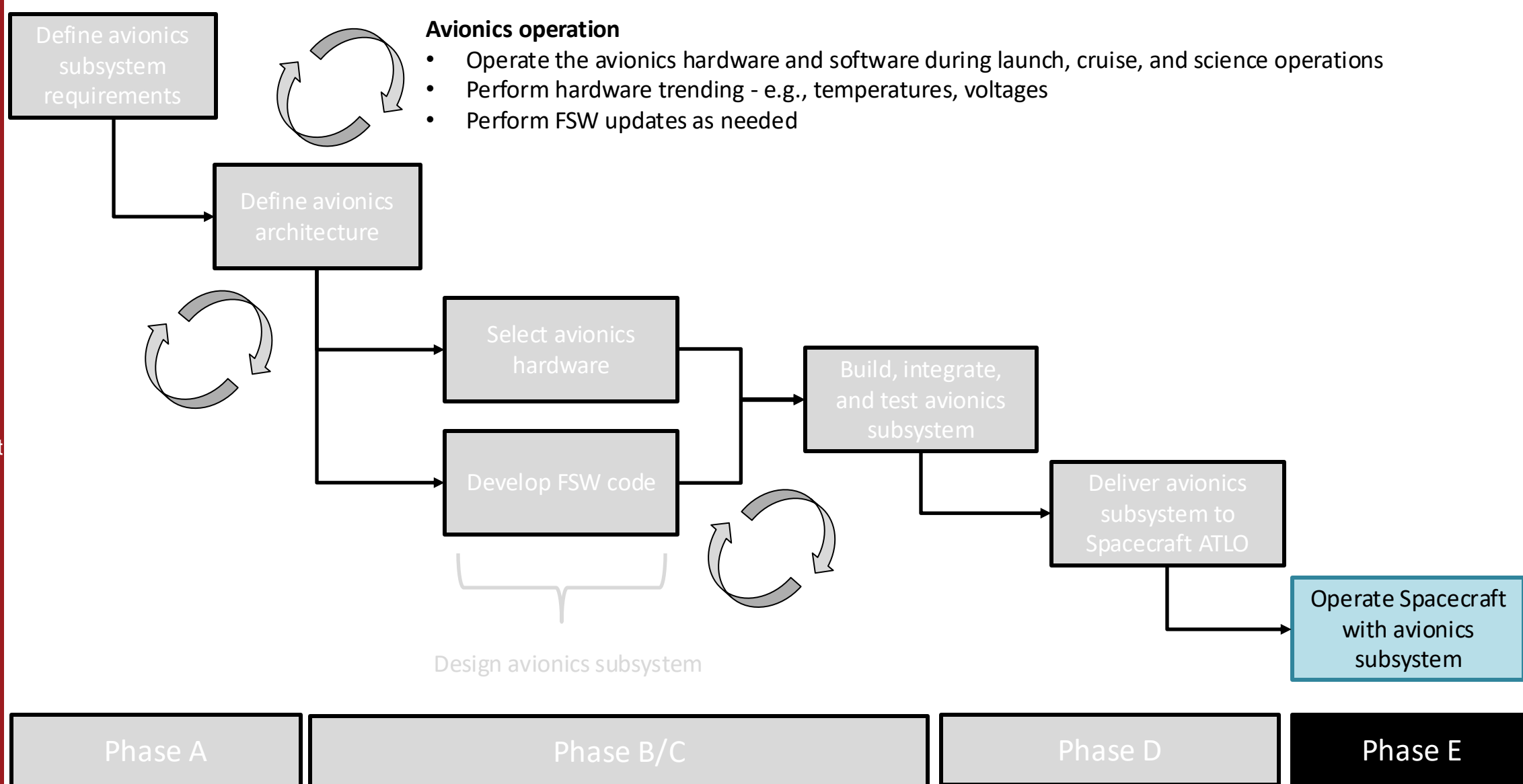
- Overview
- HW Overview
- SW Overview
- Examples

Functions

- Processing
- Intercomm
- Uplink/command
- Downlink/data
- Time mgmt.
- Autonomy & Fault mgmt

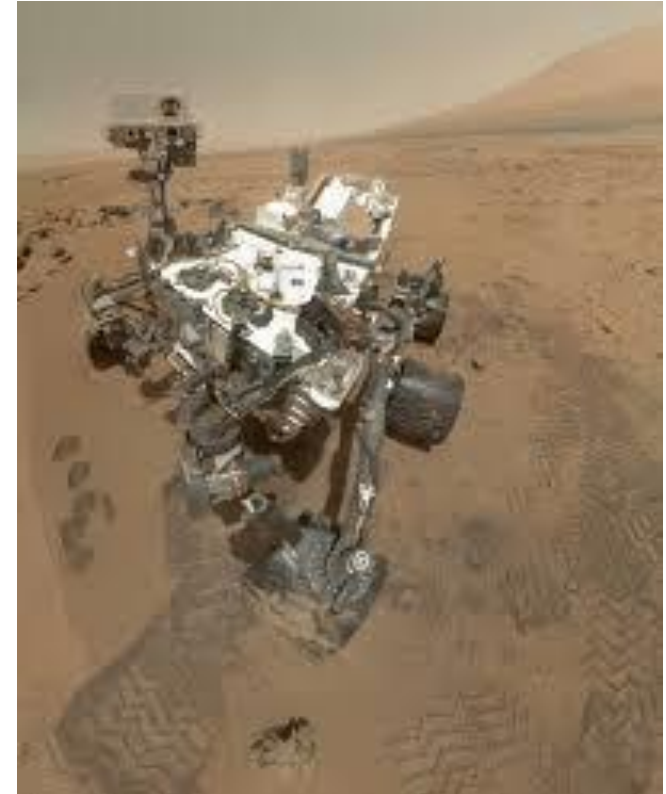
Design Process

Case Studies



Case Studies: MSL SOL 200 anomaly

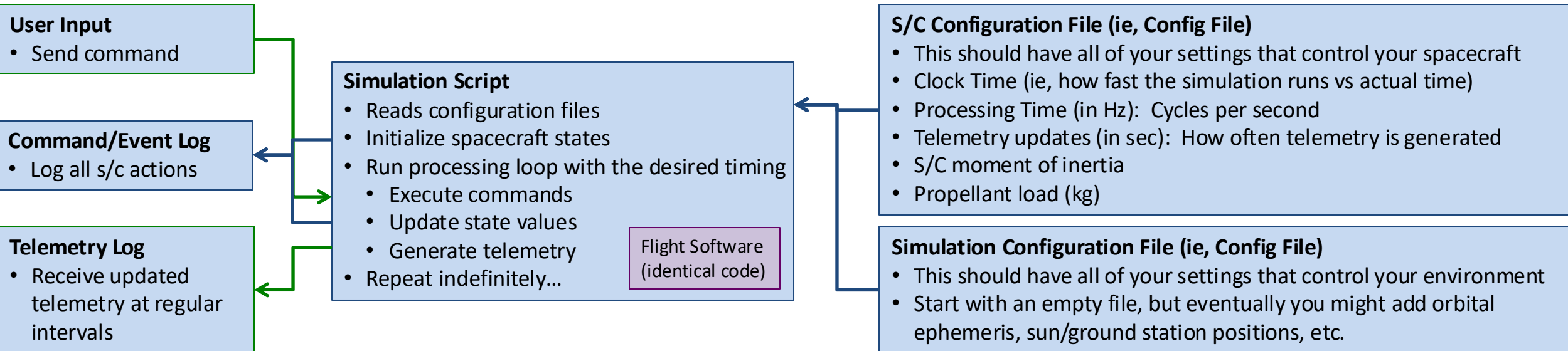
- Six months after landing on Mars, uncorrectable errors in MSL's NAND flash memory led to several FSW tasks being hung, resulting in an inability of the prime computer to turn off for its normal recharge session
- If this issue weren't resolved, the rover could "brown-out" (fully discharge its battery) within a few days, which would be unrecoverable
- Normally a watchdog timer would intervene by detecting an unresponsive FSW, but in this case enough of the FSW tasks were responsive that the watchdog timer was being reset, so the rover was not detecting a problem
- Upon noticing the errors, operators were able to send a hardware command (bypassing FSW) to swap computer sides, and regain functionality on the backup computer
- Key design features enabling recovery
 - Ability to generate telemetry and debug FSW without a NAND-based file system
 - Ability to transfer critical telemetry between prime and backup computers, and store in a location that could be read without the faulty computer's FSW
 - Ability to send hardware commands to swap computers independent of FSW
 - Segmentation of NAND across multiple chips/board for redundancy



Spacecraft Simulation Software

Spacecraft Simulation Structure

- This is the basic structure of a spacecraft simulation in Python



User Input (command-line interface)

```

> Run_SC_Sim.py TestRun-1
SC Sim is running. Enter commands:

> CDH_NOOP
>
  
```

Note: NOOP = No Operation (often used to check interface/aliveness)

Events.log

```

Wall Clock Time, SC Time (s), Event Description
20220204-133000, 0000000, Simulation started
20220204-133001, 0000001, SC configuration file read
20220204-133002, 0000002, Sim configuration file read
20220204-133200, 0000120, CDH_NOOP received
  
```

Telemetry.log (with telemetry every 5 seconds)

```

Wall Clock Time, SC Time (s), Channel, Value, units
20220204-133010, 0000010, CDH-0001, 0,
20220204-133015, 0000015, CDH-0001, 0,
20220204-133020, 0000020, CDH-0001, 0,
...
20220204-133155, 0000115, CDH-0001, 0,
20220204-133200, 0000120, CDH-0001, 1,
20220204-133205, 0000125, CDH-0001, 1,
...
  
```

Command & Telemetry (Cmds, EVRs, and EHA)

• Commands

- These are the instructions that are sent to the spacecraft that it processes and responds to.
- **There are two types**
 - **Software:** these are commands that are interpreted and executed by the flight software (FSW)
 - **Hardware (less common):** these commands bypass software and can be executed directly by the hardware (eg, HDW_SYS_RESET that resets FSW)
- **Formatting**
 - Note that the actual commands/arguments sent to the spacecraft are in binary, where each bit has a specific meaning that is defined by the command & telemetry dictionary (C&TD)
 - The C&TD typically specifies the following:
 - Command Stem: This is the mnemonic associated with the command
 - Op Code: This is the actual ID # that is sent to the s/c
 - Arguments: These are the arguments (variable # and type)
 - Argument Constraints: Argument number, types, valid types
 - Descriptions, etc
- **For our Class**
 - You do not need to translate commands to/from binary, and instead you can use & parse commands as follows:
 - Format: [subsystem]_[command mnemonic],[argument 1],[argument 2], etc.
 - Example-1: CDH_NOOP Executes no operation
 - Example-2: CDH_EHA_RATE,5 Sets EHA rate to 5 sec

• Event Records (EVRs)

- These are event records (similar to logs or text messages) that record specific events at specific times.
- These are NOT common in the industry, as many organizations will instead rely on EHA to provide this information
- **Formatting**
 - These will include the SC time and both hard-coded and variable responses
 - Format: Time, Event Description
 - Example-1 0000145, Command successfully executed: CDH_NOOP
 - Example-2: 0000147, Command successfully executed: CDH_EHA_RATE
 - Example-3: 0000148, EHA Rate set to 5 seconds
 - As with commands, these are transmitted as individual bits/codes, which are then looked up on the ground to translate into a readable description
- **For our Class**
 - Again, you do not need to translate into downlink bits.
 - All commands should include a successful (or failure) EVR, along with additional EVRs that describe the resulting changes

• Engineering, Housekeeping, & Accountability (EHA)

- This is channelized telemetry that is provided at regular intervals
 - Format: [subsystem]-[ID]: Description
 - Example-1: CDH-0001: Successful command execution counter
This counter will increment (+1) each time a command is executed
- Note that while the onboard EHA status is continually updated, these are only downlinked at regular intervals (eg, every 5-30 sec)

Software Boot / Reset Process

• Overview

- Except in rare cases, software will always be running onboard
 - Exception might a hibernation mode (eg, overnight on Mars), where there is a hardware timer that automatically boots up the software
- To ensure that this is occurring, there are various fault monitors (including watchdogs) that are monitoring software execution and will execute (via hardware) a software reset if there is an anomaly

• Boot/Reset Scenarios

- The most common boot scenario is during testing, when you are booting up the software. In this scenario, you want the software to be configured into a nominal/test state.
- During flight, the two primary (but infrequent) scenarios are:
 - Initial boot prior to launch → configures spacecraft for launch
 - This can be done as a two-step process: (1) boot into test, (2) configure for launch
 - An anomaly-induced software reset → configures spacecraft for “safe mode” that ensures hardware safety

• Boot/Reset/Mode Architecture

- There are several methods for architecting how the spacecraft responds and configures itself following a reset. This may be done via hard-coded responses or configurable tables/sequences, along with multiple possible states (or modes) depending on the spacecraft environment.
- The following example is going to describe one approach, but please keep in mind that there are many approaches

• Example: FSW Reset During Testing (prior to launch)

- During initial boot (testing or prior to launch), the spacecraft will execute the SC Configuration file that configures the spacecraft for minimum functionality
 - This includes the minimum default settings and devices powered-off except for the flight computer
 - *Good scripting practice for this example: include clear (or reinforce) variable parameters such that this file can also be used to “reset software”*
- During resets (initiated by command or onboard autonomous responses), the software will do the following:
 - Reset software (ie, run the SC Configuration file as described above)
 - Trip the FSW Reset fault monitor that looks for resets
 - If the monitor disabled, do nothing
 - If the monitor is enabled, then run the following fault response
 - » Run safe-mode configuration file

• Expected Telemetry

- EVRs
 - There should be EVRs for the FSW reset, fault monitor trip, fault monitor response, execution of the specific configuration file, and then an EVR for each of the activities within the configuration file, along with any additional hard-coded activities
- EHA
 - Spacecraft state should change (eg, from “Test” to “Safe”)
 - FSW reset and fault monitor/response counters should all increment (+1)

Example Spacecraft Commands

Subsystem	Command	Description	Expected Response
C&DH	CDH_NOOP	This command does <u>No Operation</u> (ie, nothing), which is often used to check whether there is a valid command path to the spacecraft.	EVR: Command successfully executed EHA: CDH-001 Successful command counter (+1)
C&DH	CDH_EHA_RATE,[x]	Sets the telemetry (EHA) rate for how often packets are received. Valid arguments are 5 to 1,000 sec.	EVR: EHA rate updated to x sec EHA: CDH-002 EHA Rate = x
C&DH	HWD_SYS_RESET	Resets flight software	See description in separate chart (Boot/Reset)
C&DH	CDH_UPLOSS_TIMER,[x]	Resets the uploss timer to x seconds from the present time. Note that if no commands are received within this period of time, the spacecraft will go into safe mode.	EVR: Uploss timer set to x sec EHA: CDH-003 Uploss timer = x (should be counting down to 0)
C&DH	CDH_MODE,[mode]	Sets the spacecraft mode by running an onboard configuration file that sets and/or reinforces the devices states for the relevant mode	EVR: Spacecraft mode to set [mode] EHA: CDH-004 Spacecraft Mode = [mode]
Power	PWR_DEVICE_STATE,[device],[state]	Changes state of a device. Note that this should likely be consistent with the power modes table with respect to both devices, states, and resulting power (W) values.	EVR: [device] state changed to [state] EHA: PWR-000x: [Device] State = [State] PWR-000y: [Device] Current = [current in W]

Subsystem	Command	Description	Expected Response	Comments
C&DH	[any unrecognized command stem]	Any unrecognized command stem will fail onboard command validation	EVR: Command validation failed EHA: Invalid command counter (+1)	
C&DH	[any argument that exceeds allowed range or otherwise fails]	For example, if “CDH_EHA_RATE,1” is sent, this should fail to execute.	EVR: Command execution failed EHA: Failed command counter (+1)	

SPACE SCIENCE LABORATORY UNIVERSITY OF CALIFORNIA, BERKELEY	PROCEDURE	
TITLE : Integrated Probe-Instrument LPT		THM_MINT_PROC_052_TV-LPT TEST DATE:

THEMIS PROBE-INSTRUMENT
Limited Performance Test for TVAC
LPT-TVAC
 THM_MINT_PROC_052_TV-LPT
 Revision -
 2006-AUG-03

PROBE: _____ TVAC CYCLE #: _____
 TEMP (HOT/COLD): _____

As Run on: _____ Date/Time
 By: _____ Test Conductor

Page 1

2006-Aug-04

ftp://apollo.ssl.berkeley.edu/pub/THEMIS/2%20Space%20Segment/2.2%20Spacecraft/2.2.6%20Mission%20integration%20and%20Test/Procedures/THM_MINT_PROC_052_TV-LPT.pdf

Outline of a Typical Verification Test Procedure

- **Cover Page** Project, test name, procedure # & version, etc.
- **Usage Log** Table: date, activity description, session/report #, test conductor
- **Signatures** Signatures, names, & titles of preparer & approvers
- **Revisions and Changes** Table: Rev #, Description, Date
- **Table of Contents**
- **Introduction**
 - Objective
 - Scope Description of each test and/or test case
 - Hazardous Operations Used with high electrical currents, propellant, etc.
 - Venue Specifies venue (such as which testbed, spacecraft, etc.)
- **Requirements**
 - Personnel 2-person minimum (ie, buddy system) when HW is used
 - Required Equipment
 - Support Equipment Such as GSE rack, ground system workstation, etc.
 - Special Equipment Such as break-out boxes, oscilloscopes, etc.
 - Applicable Documents Project/institutional documents (functional descriptions, interface control documents, ESD procedures, safety, etc.)
- **Special Instructions**
 - Safety
 - General Such as “The buddy system shall be used...”
 - Hardware Safety Such as descriptions of how cautions/warnings are used
 - Environmental Constraints List of applicable safety concerns (eg, ESD) Such as humidity (30-70%) and temperature
 - Special Consideration Additional notes as needed

- **Preparation**
 - Hardware Configuration Table: HW, S/N or version, comments
Additionally, any instructions on determining S/N, etc.
 - Software Configuration Table: SW, version, comments
 - Test Equipment Configuration Table: instrument, model, S/N, comments
- **Functional Procedure**
 - Test Set-up
 - Test 1: TBD
 - Test Case A
 - Test Case B
 - Etc.
 - Test n: TBD
 - Test Case A
 - Etc.
 - Shutdown
- **Appendix**
 - References and/or printouts of files used

This is where the actual test goes
(see next chart)

Additional Notes

- **Important:** Procedures are often ‘scripted’ (eg, using python) to more efficiently execute, especially for regression testing
- Most of these general sections are ‘boilerplate’ that are re-used across common tests (make sure to tailor)
- Unnecessary sections (such as for testing on a software simulation that has no HW) are typically removed left with an “N/A”

General Guidance

- Each functional test will generally include

 - Description
 - List of test cases
 - Requirements and/or verification items (VIs) being verified
 - Initial set-up
 - Test cases, each with a series of step-by-step instructions
 - Shutdown procedure

Test cases need to ensure that the overall functionality (for which the requirement is intended to address) is verified

 - For example, if the requirement is intended to all behavior related to the configuring telecom transmission rates within C&DH, then the test cases should ensure sufficient coverage

Verifying each allowed rate (eg, 10 bps, 2,000 bps, 50 kbps, 2 Mbps, etc.)

Verifying encoding matches flight ground interface control document (FG-ICD)

Verifying other configurations (eg, long vs. short frames)

Ensure adequate criteria for each test (timing, data format, etc.)

Steps need to be very specific. Depending on the test being executed, you want to specify exactly:

 - Who is doing the action, especially if there are multiple participants
 - What action is being performed (eg, send NO_OP command)
 - How is is being performed (using what user-interface, instruments, or other information)

SPACE SCIENCE LABORATORY
UNIVERSITY OF CALIFORNIA,
BERKELEY

PROCEDURE



TITLE : Integrated Probe-Instrument LPT

THM_MINT_PROC_052_TV-LPT
TEST DATE:

6.4 IDPU Power On and Test

6.4.1 ☐ Verify IDPU OCP is enabled (done by probepower). Use command pidpupwrste, if necessary.

6.4.2 ☐ Power On IDPU and Instruments with procedure: inst_leo.proc

6.4.3 ☐ Record telemetry in THM_FOP_PROC Instrument Commissioning Procedures

6.4.4 ☐ Verify that ISSR_SURVEY is incrementing (on issr page). Data will be played out later.

6.5 RF Communications System Aliveness and LPT

NOTE : Power OFF the Probe immediately in the event of a Transmitter Current Threshold Violation. Current Threshold: 1A

NOTE: Power OFF Transmitter immediately in the event that Transmitter Temperature exceeds Maximum Temperature Threshold. Monitor temperature at least once/minute until stable. Transmitter Maximum Temperature Threshold: 65° C

6.5.1 ☐ Enable Transmitter OCP, procedure: ocp_xmtr(enable).
Transmitter OCP: Required: ENABLED

6.5.2 ☐ Turn on Transmitter, procedure: xmtrpower(ON). Verify telemetry:

Telemetry	TIOS Page	Expected	Observed	Pass
Transmitter Pwr	bau_power	ENA		
Transmitter Curr	bau_transponder	0.85 ± 0.06A		
Receiver Curr	bau_transponder	0.12 ± 0.05A		
Bus Volt	bau_power	26V to 33.7V		
Batt Volt	bau_power	26V to 33.7V		
Bus Curr	bau_power	1.6A ± 0.15A		
Batt Curr	bau_power	-1.0 to 1.2A		
DC-DC Curr	bau_power	0.2 ± 0.05A		
Rcvr Temp	bau_transponder	< 60C		
Xmtr Temp	bau_transponder	< 65C		

6.5.3 ☐ Record Transponder Transmitter signal strength at RF rack.
Transmitter signal strength: Expected: -27 ± 2 dBm

6.5.4 ☐ Set Transmitter Modes:
Subcarrier mod OFF: pxmtrscaroff
Direct Carrier mod ON: pxmtrmodena
Convolutional encoding ON: pxmtrcvenon
Coherent mode ON: pxmtrcoh
Ranging OFF: pxmtrmgdis

Page 7

2006-Aug-04

Themis Example

- **Test Set-up**
 - *[Skipping step-by-step procedure to configure the testbed]*
- **Test 1: Downlink Configurations**
 - This test verifies all of the C&DH ULDL board configurations with respect to rates, encoding, and frame lengths for both X-band and UHF prime & redundant radios. Note that this test on the testbed (with FSW & EMs) is more comprehensive than a Telecom functional that will be run in ATLO on the spacecraft.
 - Test Case A: SDST-A Configurations for each data rate
 - Test Case B: SDST-B Configurations for each data rate
 - Test Cases C-F: UHF-A, UHF-B, mode changes, off-nominal configurations
 - Requirements
 - Table: Requirement #, text, status, and comments
 - *[In some cases, requirements that are overly vague will be further decomposed into individual verification items (effectively lower-level requirements, but these VIs are primarily tracked as part of this test to “sell-off” the requirement)]*
 - 5.1.1 Test Case A: SDST-A Downlink Configurations
 - Testbed on, FSW loaded (from prior set-up procedure)

5-01	TA	Disable TZ Radio interface in simulation SW	_____
		SSE → uldl tz_interface off	
5-02	TA	Enable SDST-A	_____
		SSE → uldl sdsta_interface on	
5-03	TA	Verify GSE downlink is configured for SDST-A	_____
		SSE-0101: ULDL_SDSTA_ONOFF	_____ (1 = On)

		SSE-0102: ULDL_SDSTB_ONOFF	_____ (0)
		SSE-0103: ULDL_UHFA_ONOFF	_____ (0)
		SSE-0104: ULDL_UHFB_ONOFF	_____ (0)
		SSE-0105: ULDL_TZ_ONOFF	_____ (0)
		SSE-0114: ULDL_Downlink_Source	_____ (0 = SDSTA)
5-04	TA	Send FSW command to change the downlink config	_____
		GDS → CDH_DL_SET_TX_CONFIG, LONG_SDSTA_50K	
5-05	TA	Wait for 1 min (includes max expected sync delay)	_____
		Goal is to receive ≥ 100 frames, min threshold is 3 frames for 10 bps	
5-06	TA	Using the frame accountability view, verify the following	
		TZ Downlink Configuration	_____ (LONG)
		Ant Downlink Configuration	_____ (LONG)
		LV Downlink Configuration	_____ (N/A for rates > 10K)
5-07	TA	In realtime or via post-processing, verify telemetry on SA	_____
		CDH-1010: DL_DATA_RATE	_____ (50,000 bps)
		CDH-1011: DL_FRAME_LENGTH	_____ (1000 bytes)
		CDH-1012: DL_FRAMES_SENT	_____ (≥ 100 frames)
		CDH-1013: DL_FRAMES_DROPPED	_____ (0)
		CDH-1014: DL_FRAME_COUNT	_____ (sequential)
5-08	TA	In post-processing, use gds_get_frames to verify:	_____
		TZ Downlink Rate	_____ (0 bps)
		Ant Downlink Rate	_____ (50,000 bps)
		LV Downlink Rate	_____ (0 bps)
		Etc...	
- **Note that this test is repeated for each rate on each relevant antenna**

• Test Set-up

– [Skipping step-by-step procedure to configure the test environment]

• Test 1: Downlink Configurations

– This test verifies all of the C&DH ULDL board configurations with respect to rates, encoding, and frame lengths for both X-band and UHF primes. Note that this test on the testbed is more comprehensive than a Telecom function on the spacecraft.

- Test Case A: SDST-A Configurations for each data rate
- Test Case B: SDST-B Configurations for each data rate
- Test Cases C-F: UHF-A, UHF-B, mode changes, off-nominal configurations

– Requirements

- Table: Requirement #, text, status, and comments
- [In the testbed, this will be further broken down into individual requirements. In the spacecraft, this will be further broken down into individual requirements.]

– 5.1.1 Test Case A: Downlink Configurations

- Testbed of FSW loaded (from prior set-up procedure)

- 5-01 TA Disable TZ Radio interface in simulation SW
SSE → uldl tz_interface off
- 5-02 TA Enable SDST-A
SSE → uldl sdsta_interface on
- 5-03 TA Verify GSE downlink is configured for SDST-A
SSE-0101: ULDL_SDSTA_ONOFF _____ (1 = On)

Channelized telemetry from SSE

GDS: User-interface for the Ground Data System, which sends FSW commands to the spacecraft

TA = Test Analyst. For more complex tests that require multiple individuals, this is critical.

Channelized telemetry from SC FSW (this is what SW is sending)

Commands for for execution are written-out exactly as required
SSE: User-interface for the Software Simulation Environment

- SSE-0102: ULDL_SDSTB_ONOFF _____ (0)
- SSE-0103: ULDL_UHFA_ONOFF _____ (0)
- SSE-0104: ULDL_UHFB_ONOFF _____ (0)
- SSE-0105: ULDL_TZ_ONOFF _____ (0)
- SSE-0114: ULDL_Downlink_Source _____ (0 = SDSTA)

5-04 TA Send FSW command to change the downlink config _____

GDS → CDH_DL_SET_TX_CONFIG, LONG_SDSTA_50K

5-05 TA Wait for 1 min (includes max expected sync delay) _____

Goal is to receive ≥ 100 frames, min threshold is 3 frames for 10 bps

5-06 TA Using the frame accountability view, verify the following

- TZ Downlink Configuration _____ (LONG)
- Ant Downlink Configuration _____ (LONG)
- LV Downlink Configuration _____ (N/A for rates > 10K)

5-07 TA In realtime or via post-processing, verify telemetry on SA _____

- CDH-1010: DL_DATA_RATE _____ (50,000 bps)
- CDH-1011: DL_FRAME_LENGTH _____ (1000 bytes)
- CDH-1012: DL_FRAMES_SENT _____ (≥ 100 frames)
- CDH-1013: DL_FRAMES_DROPPED _____ (0)
- CDH-1014: DL_FRAME_COUNT _____ (sequential)

5-08 TA In post-processing, use gds_get_frames to verify: _____

- TZ Downlink Rate _____ (0 bps)
- Ant Downlink Rate _____ (50,000 bps)
- LV Downlink Rate _____

Verifying what the ground sees (frames, timing, formatting, etc.)

– Note that this test is repeated for each rate on each prime

- **Test Set-up**

– [Skipping step-by-step procedure to configure the testbed]
- **Test 1: Downlink Configurations**
 - This test verifies all of the C&DH ULDL board configurations with respect to rates, encoding, and frame lengths for both X-band and UHF prime & redundant radios. Note that this test on the testbed (with FSW & EMs) is more comprehensive than a Telecom functional that will be run in ATLO on the spacecraft.
 - Test Case A: SDST-A Configurations for each data rate
 - Test Case B: SDST-B Configurations for each data rate
 - Test Cases C-F: UHF-A, UHF-B, mode changes, off-nominal configurations
 - Requirements
 - Table: Requirement #, text, status, and comments **Not Passed**
 - [In some cases, requirements that are overly vague will be further decomposed into individual verification items (effectively lower-level requirements, but these VIs are primarily tracked as part of this test to “sell-off” the requirement)]
 - 5.1.1 Test Case A: SDST-A Downlink Configurations
 - Testbed on, FSW loaded (from prior set-up procedure)
 - 5-01 TA Disable TZ Radio interface in simulation SW **SSE → uldl tz_interface off** ✓
 - 5-02 TA Enable SDST-A **SSE → uldl sdsta_interface on** ✓
 - 5-03 TA Verify GSE downlink is configured for SDST-A **SSE-0101: ULDL_SDSTA_ONOFF** 1 (1 = On) ✓

	SSE-0102: ULDL_SDSTB_ONOFF	<u>0</u>	(0)	
	SSE-0103: ULDL_UHFA_ONOFF	<u>0</u>	(0)	
	SSE-0104: ULDL_UHFB_ONOFF	<u>0</u>	(0)	
	SSE-0105: ULDL_TZ_ONOFF	<u>0</u>	(0)	
	SSE-0114: ULDL_Downlink_Source	<u>0</u>	(0 = SDSTA)	
5-04	TA Send FSW command to change the downlink config			✓
	GDS → CDH_DL_SET_TX_CONFIG, LONG_SDSTA_50K			
5-05	TA Wait for 1 min (includes max expected sync delay)			✓
	Goal is to receive ≥ 100 frames, min threshold is 3 frames for 10 bps			
5-06	TA Using the frame accountability view, verify the following			X
	TZ Downlink Configuration	SHORT	(LONG)	
	Ant Downlink Configuration	SHORT	(LONG)	
	LV Downlink Configuration	N/A	(N/A for rates > 10K)	
5-07	TA In realtime or via post-processing, verify telemetry on SA			✓
	CDH-1010: DL_DATA_RATE	50,000	(50,000 bps)	
	CDH-1011: DL_FRAME_LENGTH	1,000	(1000 bytes)	
	CDH-1012: DL_FRAMES_SENT	1,254	(≥ 100 frames)	
	CDH-1013: DL_FRAMES_DROPPED	<u>0</u>	(0)	
	CDH-1014: DL_FRAME_COUNT	YES	(sequential)	
5-08	TA In post-processing, use gds_get_frames to verify:			✓
	TZ Downlink Rate	<u>0</u>	(0 bps)	
	Ant Downlink Rate	50,000	(50,000 bps)	
	LV Downlink Rate	<u>0</u>	(0 bps)	
	Etc...			

– Note that this test is repeated for each rate on each relevant antenna

- **Test Files**

- Original signed procedure, as-run procedure, and any files used as part of the test (eg, command sequences)
- Session report

- **Software Simulation Environment**

- Data varies depending on the SW/HW, but it can be similar to spacecraft telemetry (eg, channelized EHA)

- **Spacecraft Telemetry**

- Note that when the GDS unpacks the data, it adds several fields to the data, such as:
 - Session ID, Testbed, Record Type, Received Time (or Earth Return Time, ERT; typically in YYYY-DDDTHH:MM:SS.SSSSSS format; see ISO 8601)
 - For EHA, an “alarm state” might also be added, which flags EHA values that exceed specified caution (yellow) and warning (red) thresholds
- EHA (common)
 - EHA Channel ID, Name, and Module (eg, CDH-0001, Successful command execution counter, uplink)
 - DN (design number; that is, the raw value that is received from the spacecraft)
 - EU (enumerated number; that is, the converted number that makes sense. For example, 0 DN = “Off”)
 - SCLK (spacecraft clock in Unix Time that an event was recorded; that is seconds since midnight 1/1/1970, including subseconds)
 - SCET (spacecraft event time, which is sclk converted to a more typical ISO 8601 format; see <https://www.epochconverter.com/>)
- EVRs (less common)
 - EVR Name and Message
 - EVR Type (eg, diagnostic, warnings, etc.)
 - Sequence ID (per EVR type, which allows insight into any EVRs that were not received)
 - SCLK & SCET
- Spacecraft Files (eg, tables)
 - Additionally, custom files, such as engineering files (parameter tables) or science data (images) may be downlinked as well
 - Hardware or simulated models will often have the ability to generate test files